Saras — MVP Build Pack v2.0

Everything needed to start implementation: product, RBAC, workflows, architecture, contracts, UX, security, engineering, delivery

15 October 2026 · implementation starts 16 October 2026

Contents

00 — Start Here
- Reading order
- Machine-readable artifacts
- The ten rules every engineer must know

01 — Product Brief & YC Narrative
- 1.1 One-liner
- 1.2 The problem
- 1.3 The solution
- 1.4 Why now
- 1.5 Who it's for (initial customer)
- 1.6 Business model (proposed)
- 1.7 Competition and how we're different
- 1.8 What exists today and what ships next
- 1.9 Three-minute demo script (for the YC video)
- 1.10 Metrics we will report
- 1.11 Risks and how we address them

02 — Product Requirements (PRD)
- 2.1 Goals for the MVP
- 2.2 Non-goals for the MVP
- 2.3 Personas
- 2.4 Feature specifications
- 2.5 Non-functional requirements
- 2.6 Analytics events (PostHog)

03 — Organization Hierarchy & Role-Based Access Control
- 3.1 Design stance
- 3.2 How an MNC is organized, and how Saras maps it
- 3.3 Four permission planes
- 3.4 Project capability catalogue (enforced by the system)
- 3.5 Role templates for MNC and startup teams
- 3.6 Agent plane: what the AI teammate may do
- 3.7 The five AI roles and where each lives
- 3.8 Permission evaluation (exact algorithm)
- 3.9 Access lifecycle (joiner, mover, leaver, guest)
- 3.10 Separation of duties and enterprise controls (MVP)
- 3.11 Worked examples

04 — App Workflows & User Journeys
- 4.1 End-to-end flow
- 4.2 Journey A — Organization onboarding (Org Owner)
- 4.3 Journey B — Team and project setup (Team Owner → Project Owner)
- 4.4 Journey C — Daily collaboration in the Arena
- 4.5 Journey D — Private → Arena (the bridge)
- 4.6 Journey E — SDLC loop and gates

4.7 Journey F — AI teammate (AI as employee)
4.8 Journey G — Routines (AI as routine tasker)
4.9 Journey H — Oversight and access (Unit Lead)
4.10 Journey I — People lifecycle
4.11 State machines (implementation reference)
4.12 Notification matrix

05 — Architecture (HLD + LLD, consolidated)
5.1 Shape of the system
5.2 Technology stack (final for MVP)
5.3 Monorepo layout
5.4 API module map (bounded contexts)
5.5 Request lifecycle (LLD)
5.6 Realtime
5.7 Runtime adapter (inside `api` + `worker`)
5.8 App Tools MCP server
5.9 Jobs and schedules (pg-boss)
5.10 Caching
5.11 Errors and resilience
5.12 Deployment topology
5.13 Capacity estimate (year 1)
5.14 What we'd change at scale

06 — Data Model
6.1 Entity relationships
6.2 Table catalogue
6.3 Invariants enforced by the database
6.4 Key rules not expressible in SQL (enforced in `packages/domain`)
6.5 Indexing and partitioning
6.6 Retention
6.7 Migrations

07 — API Reference
7.1 Conventions
7.2 Resource map
7.3 Worked examples
7.4 Error codes
7.5 Domain events (outbox → pg-boss consumers)
7.6 Endpoint catalogue with authorization rules (generated)

08 — Agent Runtime Integration (Tau)
8.1 Boundary
8.2 RIC v0.2 at a glance
8.3 Turn modes
8.4 App Tools v0.2 (20)
8.5 Autonomy enforcement (two layers)
8.6 Stub runtime (app-team owned)
8.7 Conformance and isolation certification
8.8 Integration checkpoints with the Tau team

09 — UX Specification & Design System
9.1 Information architecture
9.2 Screen inventory (canvas artboards)
9.3 Design tokens
9.4 Component inventory
9.5 Interaction rules
9.6 Copy guidelines
9.7 Accessibility and responsive

10 — Security, Privacy & Compliance
10.1 Security principles
10.2 Threat model (top risks)
10.3 Identity and access
10.4 Data protection
10.5 Application security controls
10.6 Privacy
10.7 Compliance roadmap
10.8 Security checklist per feature (Definition of Done)

11 — Engineering Handbook
- 11.1 Day-1 setup
- 11.2 Coding standards
- 11.3 Git and review
- 11.4 Definition of Done (every story)
- 11.5 Testing strategy
- 11.6 CI/CD
- 11.7 Environments and configuration
- 11.8 Observability and SLOs
- 11.9 Runbooks (summary)

12 — Delivery Plan, YC Demo Slice & Backlog
- 12.1 Timeline
- 12.2 YC demo slice — day by day (16–30 Oct)
- 12.3 Team and ownership
- 12.4 Ceremonies
- 12.5 Risks
- 12.6 Launch checklist (GA)
- 12.7 Decisions to confirm with founders (defaults used in this pack)

13 — Decision Log (ADRs, consolidated)

14 — Glossary

00 — Start Here

Saras moves AI from private chats into a shared team space. Humans and AI work together across the whole development cycle. The AI acts as knowledge base, brainstormer, employee, agent and routine tasker. Teams control it: they define their own roles, approvals and how much the AI may do. Organizations of any size, up to MNCs, get structure, oversight and safety without taking control away from teams.

This Build Pack is the **single source of truth** for the MVP. It supersedes the v1.0 Architecture Playbook and the v1.1 Implementation Plan wherever they differ (decision log, doc 13).

Reading order

#	Document	Read if you are…
01	Product Brief & YC Narrative	Everyone. YC application material
02	Product Requirements (PRD)	Everyone. Feature specs F01–F21, non-goals, NFRs
03	**Organization Hierarchy & RBAC**	Everyone. MNC mapping, four permission planes, 27 capabilities, templates, agent autonomy
04	App Workflows & User Journeys	Product, design, engineering
05	Architecture	Engineering
06	Data Model	Backend
07	API Reference	Backend, frontend
08	Agent Runtime Integration (Tau)	Backend, Tau team
09	UX Specification & Design System	Design, frontend
10	Security, Privacy & Compliance	Everyone touching data or permissions
11	Engineering Handbook	Engineering. **Day-1 setup is here**
12	Delivery Plan, YC Demo Slice & Backlog	Everyone. **What we build tomorrow**
13	Decision Log	Leads
14	Glossary	Everyone

Machine-readable artifacts

File	What	Verified
`contracts/schema_v2.sql`	PostgreSQL 16 schema: 45 tables, RLS, RBAC, routines	Applied to PG 16.15 + pgvector; RBAC smoke tests pass (doc 06 §6.3)
`contracts/openapi_v2.yaml`	REST API: 132 operations, permission rule on each	Redocly lint: 0 errors
`contracts/runtime-integration-contract_v0.2.yaml`	App ↔ Tau contract	Redocly lint: valid
`contracts/app-tools_v0.2.mcp.json`	20 App Tools for the runtime, with autonomy classes	Parsed
`contracts/role_templates_v2.json`	7 role templates (roles, capabilities, gates, agent policy)	Every approval gate's roles hold the matching capability (script-checked)
`Saras_MVP_Backlog_v2.xlsx`	77 stories, 365 points, phase / track summary, capacity	Formulas recalculated, 0 errors
`diagrams/`	16 diagrams (Mermaid source + SVG + PNG)	Rendered
Design canvas "Saras App UI"	14 artboards, light and dark	—

The ten rules every engineer must know

1. **Teams own their structure.** Roles are team-defined bags of 27 fixed capabilities. Gate approvers are roles.
2. **Permissions follow responsibility, not seniority.** Job level never grants access.
3. **Oversight ≠ access.** Org and unit dashboards show status only. Content requires project membership (Observer at least).
4. **Private is private.** The bridge (bind, pull, push) is the only crossing. Push copies one artifact, never the session.
5. **One Arena per project, one agent session per Arena.** A private session binds to ≤1 project, forever.
6. **AI never exceeds the human it acts for.** Autonomy policy ∩ org ceiling ∩ invariants, enforced in App Tools.
7. **Humans vote on gates and verify tasks.** Agents never do. Separation-of-duties rules apply.

8. **The database is the last line.** FORCE RLS for tenant, Private and Arena visibility, plus capability-checked Arena writes.
9. **Contracts first.** Change `packages/contracts` before code. The runtime is reached only through RIC v0.2.
10. **Everything consequential is audited.** Append-only, hash-chained.

01 — Product Brief & YC Narrative

> Build Pack v2.0 · Owner: Founders · Use: YC application, investor conversations, onboarding new hires Facts not yet known are left as **[placeholders]**. Fill them in; do not invent numbers.

1.1 One-liner

Saras is the shared workspace where software teams and AI work together across the entire development cycle. Today people use AI alone, in private chats. Saras moves that work into a space the whole team shares.

Alternatives for the application form: - “Multiplayer AI for software teams: one shared space per project where humans and an AI teammate plan, build and ship together.” - “We turn private AI chats into a team’s shared, governed AI teammate across the full SDLC.”

1.2 The problem

1. **AI use at work is private and fragmented.** Every engineer, PM and designer has their own assistant. Ideas, specs and decisions live in individual chat histories. The team doesn’t see them, can’t build on them, and can’t trust where they came from.
2. **The work AI helps with is collaborative.** Requirements, design, implementation, review, release and operations need handoffs, approvals and shared context. Today those happen in a different set of tools than the AI.
3. **Companies can’t let AI loose on team work without control.** Large organizations need roles, approvals, audit and isolation between projects before AI can act on shared work. Most AI tools are built for one person, not for a team with structure.

1.3 The solution

Saras surface	What it gives the team
Private space (per person, available everywhere)	Your own AI workspace: brainstorm, draft, research. Private by default
Bridge	Push the useful result (a goal, spec or plan) into your team’s Arena in one click, with preview. The private conversation is never shared
Arena (one per project)	The team’s shared space: humans plus a central AI agent in **one continuous session** that knows the project
Team-defined roles	Each team creates the roles it actually has (Frontend dev, Architect, Security champion…) and decides who approves each stage. No fixed org chart is imposed
SDLC loop	PLAN → DESIGN → BUILD → TEST → DEPLOY → MAINTAIN → next cycle, with stage artifacts and human approval gates. It runs for as long as the product is operated
AI in five roles	Knowledge base, brainstormer, employee (works assigned tasks), agent (uses tools), routine tasker (digests, standups, nudges). All of it governed by the team’s autonomy settings
Organization oversight	Leaders see status across teams (stages, progress, blockers) without reading team conversations

1.4 Why now

- Teams already use AI daily for individual work. The next step is teams working *with* AI, the way they moved from email to shared channels and from local docs to shared docs.
- Agent harnesses (e.g. the open-source Pi coding agent, which our Tau runtime forks) are now capable enough to do real multi-step work. The missing layer is collaboration, permissions and governance around them.
- Enterprises are deciding how AI may act on shared work. A product that bakes in roles, approvals, isolation and audit from day one can be adopted team by team inside large companies.

1.5 Who it’s for (initial customer)

Segment	Why they buy	Entry point
Primary: product engineering teams (5–50 people) inside mid-size companies and MNCs	Already use AI individually. Need shared context and approvals. Can adopt without a company-wide mandate	Team-led: one team sets up a project and invites others

Segment	Why they buy	Entry point
Secondary: startups (10–100 people)	Want speed with lightweight structure	Org signs up; every team gets Arenas
Later: regulated enterprises	Need governance, audit, data residency	Enterprise preset, SSO/SCIM, BYO runtime (post-MVP)

Adoption motion: "controlled by the teams that need it." A team starts. Its org admin connects SSO when more teams join. Leaders get dashboards for free.

1.6 Business model (proposed)

Plan	Who	Pricing basis
Team	One or a few teams	Per active member / month + included agent usage
Organization	Many teams, units, SSO/SCIM, dashboards	Per member / month, usage pooled, budgets
Enterprise	Regulated, large	Annual contract, data residency, BYO runtime/model keys

Prices: **[to be set after design-partner interviews]**. Agent usage (model cost) is metered per project and capped by org budgets from day one (doc 05, doc 10).

1.7 Competition and how we're different

Positioning by category, not feature-by-feature claims:

Category	Strength	Gap Saras fills
Personal AI assistants & AI IDEs	Excellent for one person	Private by design. No team space, roles, approvals or SDLC loop
Autonomous coding agents	Do multi-step coding tasks	Agent-centric, not team-centric. Little say for the team's own structure
Team chat with AI add-ons	Shared conversation	No SDLC structure, stage artifacts, gates or per-project agent session
Project management / issue trackers with AI	Planning and tracking	AI is a feature, not a teammate that works in the shared session across stages

Our wedge: the team's own structure (custom roles + gates) combined with a shared AI session per project and a Private → Arena bridge. Isolation and audit are built in, so large companies can let it in.

1.8 What exists today and what ships next

Asset	Status
Product & architecture documentation (this Build Pack)	Done
Clickable UI design (Private, Arena, setup wizard, loop, dashboards, inbox, onboarding, RBAC screens)	Done, on design canvas
Database schema (PostgreSQL, RLS, RBAC)	Done, validated against PostgreSQL 16
API contract (OpenAPI 3.1, permission rule per endpoint)	Done, lint-clean
Runtime contract with Tau (our Pi-based agent harness)	Done (v0.2)
YC demo slice (working vertical slice on staging)	**Building, 16–30 Oct 2026**
MVP GA	**Target 22 Feb 2027**
Users / design partners	**[number and names when signed]**

1.9 Three-minute demo script (for the YC video)

Time	Show	Say
0:00	Private home	"This is where everyone uses AI today, alone. Saras keeps that, and adds the team."
0:20	Create project → pick "Enterprise product squad" template → rename a role, add "Security champion" → DEPLOY needs 2 approvers	"Every team defines its own roles and who approves what. Nothing is imposed."
0:50	Arena opens. The central agent greets with the team's roles	"One shared space per project, with one AI that knows the whole project."

Time	Show	Say
1:10	Private session: “draft goal.md from our discussion” → Push to Arena → PLAN gate requested	“Personal work becomes team work in one click, and only the artifact is shared.”
1:40	Approver votes in the Arena. Loop moves to DESIGN. Routine digest posts	“Humans make the calls. The AI does the routine work.”
2:10	Assign a task to the AI teammate → draft lands in review	“The AI works as a teammate, inside the team’s rules.”
2:35	Org dashboard: status across teams, no conversations visible	“Leaders get visibility, teams keep ownership.”
2:50	Close	“Saras: AI moves from private chats into the team.”

1.10 Metrics we will report

Metric	Definition
Weekly active Arenas	Projects with ≥3 human posts and ≥1 agent turn in a week
Bridge rate	Pushes from Private to Arena per weekly active user
Loop velocity	Median time from PLAN to DEPLOY per cycle
Agent work accepted	% of AI-teammate tasks verified without rework
Team expansion	New teams per org per month after the first
Retention	Week-4 retention of teams that complete project setup

1.11 Risks and how we address them

Risk	Mitigation
Agent quality not good enough for real team work	Humans gate every stage. The AI drafts and humans approve. Runtime quality is improved separately (Tau) without blocking the app
Enterprises block AI on shared data	Isolation per project, Private never shared, org autonomy ceilings, audit, data region
Teams don’t change habits	Private stays familiar (same as their current AI use). The bridge is one click. Routines give value without behaviour change
Model cost	Per-org budgets, batching of ambient messages, model classes per task

02 — Product Requirements (PRD)

> Build Pack v2.0 · Owner: Product · Status: **Frozen for MVP** (changes via decision log, doc 13) Stories with acceptance criteria and estimates: `Saras_MVP_Backlog_v2.xlsx` (77 stories, 365 points). Feature IDs (F01–F21) match epic codes there.

2.1 Goals for the MVP

1. A team can set up a project **with its own roles and approval rules** in under 10 minutes.
2. The team collaborates in **one Arena per project** with a central AI agent that has the whole project's context.
3. Individuals keep their **Private space** and move results into the Arena through the **bridge**, without exposing private conversations.
4. Work moves through a **repeating SDLC loop** with human gates and stage artifacts.
5. AI acts as **knowledge base, brainstormer, employee, agent and routine tasker** within team-set autonomy.
6. An MNC can adopt it safely: **org hierarchy, org roles, SSO, guests, dashboards without content access, audit**.

2.2 Non-goals for the MVP

Not in MVP	Why / when
Voice input	Scope; post-MVP
Developer mode / extension marketplace	Depends on Tau extension model; post-MVP
Multiple channels inside an Arena	Single Arena by design
Parallel development cycles per project	One active cycle, others operating; post-MVP
Agents posting as a person ("reply on my behalf")	Trust model needs usage data; post-MVP
Code execution in sandboxes, automated PR review and deploys	Tau runtime phase (post-MVP P7)
Monitoring-triggered cycles (alerts → new cycle)	Post-MVP P8
Mobile apps, Slack/Teams integrations, email replies	Post-MVP
Multiple data regions, BYO model keys, BYO runtime	Enterprise phase

2.3 Personas

Persona	Typical title	Main jobs in Saras
Org Owner / Admin	Founder, CIO office, eng-productivity lead	Create org, connect SSO, structure units, set policies, upload org knowledge
Security & Compliance Admin	CISO office	Set guest/SSO/agent-ceiling policies, audit, exports
Unit Lead	VP, Director	Watch portfolio status, request access when needed
Team Owner / Admin	Engineering manager, team lead	Create teams and projects, manage people
Project Owner	EM, PM, program manager	Run setup (roles, gates, autonomy, routines), manage access
Contributor	Engineer, designer, QA, PM, SRE	Private work, Arena collaboration, tasks, push artifacts, vote on gates their role owns
Observer	Stakeholder, auditor, exec	Read-only Arena
Guest	Contractor, vendor	Contributor in specific projects, time-bound

2.4 Feature specifications

Each feature lists scope, key rules and acceptance criteria (AC). "Must" = MVP launch blocker.

F01 Authentication & SSO — Must

- **Scope:** WorkOS AuthKit login (Google, Microsoft, email magic link); enterprise SAML/OIDC via WorkOS admin portal; session management; logout everywhere on offboarding.
- **AC:**
  - Login → callback → session cookie (HttpOnly, Secure, SameSite=Lax).
  - When `require_sso` is set, non-SSO login for that org's domain is refused with a clear message.

    - Offboarding revokes all sessions within 60 s.

F02 Organization, presets & policies — Must

- **Scope:**
    - Create org (Startup / Enterprise preset).
    - Org profile.
    - Org policies (doc 03 §3.10).
    - Org knowledge upload (principles, guidelines, ways of working, moat) for the central agent.
- **AC:**
    - Creator is Org Owner.
    - Presets fill `org_policies`.
    - Security fields are editable only by Owner / Security admin.
    - Every change is audited.

F03 Org hierarchy & unit roles — Must

- **Scope:** units tree (≤5 levels, custom level names); Unit Lead and Unit Admin assignments; move teams between units.
- **AC:**
    - A move that would create a cycle is rejected.
    - Units can be deleted only when empty.
    - Unit roles apply to the whole subtree.

F04 Members, invites, guests, offboarding — Must

- **Scope:**
    - Invites (org, team, project).
    - Directory with search.
    - Org role changes.
    - Guests with expiry.
    - Offboarding.
    - SCIM sync (P1 within MVP window).
- **AC:**
    - Guests must have `guest_access_until`, see only co-members, and are suspended at expiry (warned 7 days before).
    - Offboarding removes team and project memberships.
    - Owner-less projects are flagged.
    - Private content is deleted after the retention period.

F05 Teams — Must

- **Scope:** create team (per policy), team roles (Owner / Admin / Member), unit assignment, team page, team defaults (template, project creation permission).
- **AC:**
    - Every team keeps ≥1 owner.
    - Team creation is blocked when policy forbids it, with a message naming who can create teams.

F06 Org & unit dashboards — Must

- **Scope:**
    - KPI tiles.
    - Project table (team, cycle, stage, loop pips, progress, members, attention).
    - Stage distribution.
    - Filters by unit and team.
- **AC:**
    - Responses contain **no** message, artifact or task content (contract test).
    - A Unit Lead sees only their subtree.
    - Loads in < 500 ms for 500 projects.

F07 Project setup wizard — Must (USP)

- **Steps:**
    1. Basics.
    2. Members.
    3. Roles (7 templates or blank; edit names and capabilities).
    4. Assign members to roles.

5. Stage gates (mode, approver roles, min approvals, distinct roles, no-self-approval, required artifact; MAINTAIN cycle openers).
6. Agent autonomy and starter routines.
7. Review and create.

- **AC:**
  - Validation blocks create when:
    - there is no owner;
    - an approval gate has no staffed role;
    - `min_distinct_roles` is greater than the number of staffed approver roles;
    - autonomy is above the org ceiling.
  - Create is atomic.
  - The Arena opens with a central-agent welcome that lists roles and gates.

F08 Project roles, members & access — Must

- **Scope:**
  - Manage members, roles and custom roles after setup.
  - Observer role.
  - Discoverability (hidden / team / org).
  - Access requests.
  - Agent autonomy settings.
- **AC:**
  - A role cannot be deleted if a gate would become unstaffed.
  - The last owner cannot be removed.
  - Observers cannot post (UI and database).
  - Member limit is enforced.
  - Cross-unit additions respect org policy.

F09 Arena — Must

- **Scope:**
  - One Arena per project.
  - Single message stream with threads (replies).
  - `@person`, `@role`, `@central` tags.
  - Attachments, artifact references.
  - Realtime presence, typing and streaming.
  - Loop strip, approval cards, right panel (overview, tasks, artifacts, members).
- **AC:**
  - Messages arrive to all online members p95 < 300 ms.
  - `@role` notifies every holder (requires `arena.tag_roles`).
  - Agent and event messages are visually distinct.

F10 Central agent (knowledge base & agent) — Must

- **Scope:**
  - One runtime session per Arena, created at setup.
  - `@central` turns with streaming, citations and a step drawer.
  - Ambient messages batched as observations.
  - System notices on stage changes.
- **AC:**
  - The agent only reads its own project, verified by the canary suite.
  - Failures show “agent unavailable”; human chat keeps working.

F11 Routines (routine tasker) — Must

- **Scope:**
  - Templates: daily digest, standup collector, gate nudge (pending > N hours), weekly progress report, custom prompt.
  - Cron scheduling (≥1 h), timezone, pause, run now, last-run status.
- **AC:**
  - Exactly one run per schedule slot.
  - Output posts to the Arena as the central agent and respects the `post` autonomy level.
  - When the runtime is down, digest and nudge fall back to deterministic templates.

F12 Private space & personal agent — Must

- **Scope:** Private home, sessions, recent list, search own sessions, model class selector, attachments, artifacts library, dock on every screen.
- **AC:**
  - No other user, admin or agent session can read Private content (verified by RLS tests).
  - Sessions show their project binding when bound.

F13 Bridge & Push to Arena — Must

- **Scope:**
  - Bind a session to one project (write-once).
  - Pull a read-only context snapshot.
  - Push an artifact version with preview: new vs new version, attach to the active cycle, request gate, notify.
- **AC:**
  - A push never copies the session.
  - Lineage is recorded.
  - The source session binds on first push.
  - Pulling context from a different project returns `BRIDGE_SCOPE_MISMATCH`.

F14 Artifacts — Must

- **Scope:**
  - Versioned markdown and uploaded files.
  - Statuses (draft, in_review, approved, superseded).
  - Diff, export.
  - Stage artifacts per cycle.
- **AC:**
  - Content is deduplicated by sha256.
  - One approved version per artifact.
  - Export requires `artifact.export`.

F15 Tasks, delegation & AI teammate — Must

- **Scope:**
  - Tasks with weight, acceptance criteria and dependencies.
  - Delegate to a person or role.
  - Assign to the AI teammate with a required human reviewer.
  - Verify to complete.
- **AC:**
  - Only humans with `task.verify` complete tasks (DB check).
  - AI-teammate tasks move at most to *In review*.
  - Progress % uses the weighted formula.

F16 SDLC loop — Must

- **Scope:**
  - Cycles (one active, others operating).
  - Stages PLAN → MAINTAIN.
  - Gate requests, votes with separation-of-duties, send-back with reason, decommission.
  - Evidence (GitHub commit of approved stage artifacts, PR/CI/deploy links).
  - `progress.md`.
- **AC:**
  - No double advance under concurrent votes.
  - The required artifact must be approved before the gate can pass.
  - Every transition is append-only with actor and reason.

F17 Notifications & DMs — Must

- **Types:** artifact push, stage update, delegation, tag, DM, approval request, access request, assignment, routine.
- **Scope:** inbox with filters, “needs action”, preferences, realtime toasts; DMs 1:1 and small groups. Email digest is P1.
- **AC:**
  - Every notification deep-links to its source.
  - Nobody is notified about a project they can’t read.

F18 Search — Must

- **Scope:** projects, messages, artifacts, tasks, people, own private sessions.
- **AC:** results are ACL-filtered in SQL; p95 < 400 ms.

F19 Audit & compliance — Must

- **Scope:** append-only, hash-chained audit log of medium- and high-risk actions; browse; export.
- **AC:** every RBAC change, gate vote, push, access decision, policy change and offboarding is audited.

F20 Runtime integration — Must

- **Scope:**
  - Runtime adapter (RIC v0.2).
  - App Tools MCP server (20 tools).
  - Stub runtime (scripted and thin-LLM).
  - Canary isolation suite.
  - Tau swap.
  - Usage and budgets.
- **AC:**
  - Conformance ≥95% and canary 100% on the GA runtime.
  - Budgets enforced with alerts at 80% and 100%.

F21 Platform foundations — Must

- **Scope:** infrastructure, CI/CD, migrations, observability, design system, feature flags, analytics.
- **AC:** see doc 11 Definition of Done.

2.5 Non-functional requirements

Area	Requirement
Availability	99.9% monthly for API and realtime (agent features depend on runtime and model providers; degrade gracefully)
Latency	API p95 < 250 ms reads / 500 ms writes; Arena fan-out p95 < 300 ms; agent first event p95 < 3 s
Scale (MVP year 1)	100 orgs, 5,000 users, 2,000 projects, 200k messages/day
Security	Tenant isolation (RLS), Private isolation, project isolation for agents, SSO, audit (doc 10)
Accessibility	WCAG 2.1 AA
Browsers	Latest 2 versions of Chrome, Edge, Safari, Firefox; responsive down to 390 px width for reading and approving
Data	Region ap-south-1; RPO ≤ 5 min, RTO ≤ 1 h
Cost	Model spend metered per project, capped by org budgets

2.6 Analytics events (PostHog)

`org_created`, `team_created`, `project_setup_started`, `project_setup_completed` (template, roles_count, custom_roles_count), `arena_message_posted` (is_agent), `central_invoked`, `private_session_started`, `artifact_pushed`, `gate_requested`, `gate_decided`, `cycle_opened`, `task_assigned_to_agent`, `task_verified`, `routine_created`, `routine_run_posted`, `access_requested`, `dashboard_viewed`.

Rules: - Events carry ids only. - No message or artifact text. - No email addresses.

03 — Organization Hierarchy & Role-Based Access Control

> Build Pack v2.0 · Owner: Tech lead · Status: **Approved for implementation** · Supersedes: v1.0 doc 03 §3.4 and v1.1 plan §2.1–2.2
> Contracts: `contracts/schema_v2.sql` (tables, RLS), `contracts/openapi_v2.yaml` (`x-required-permission` on every operation)

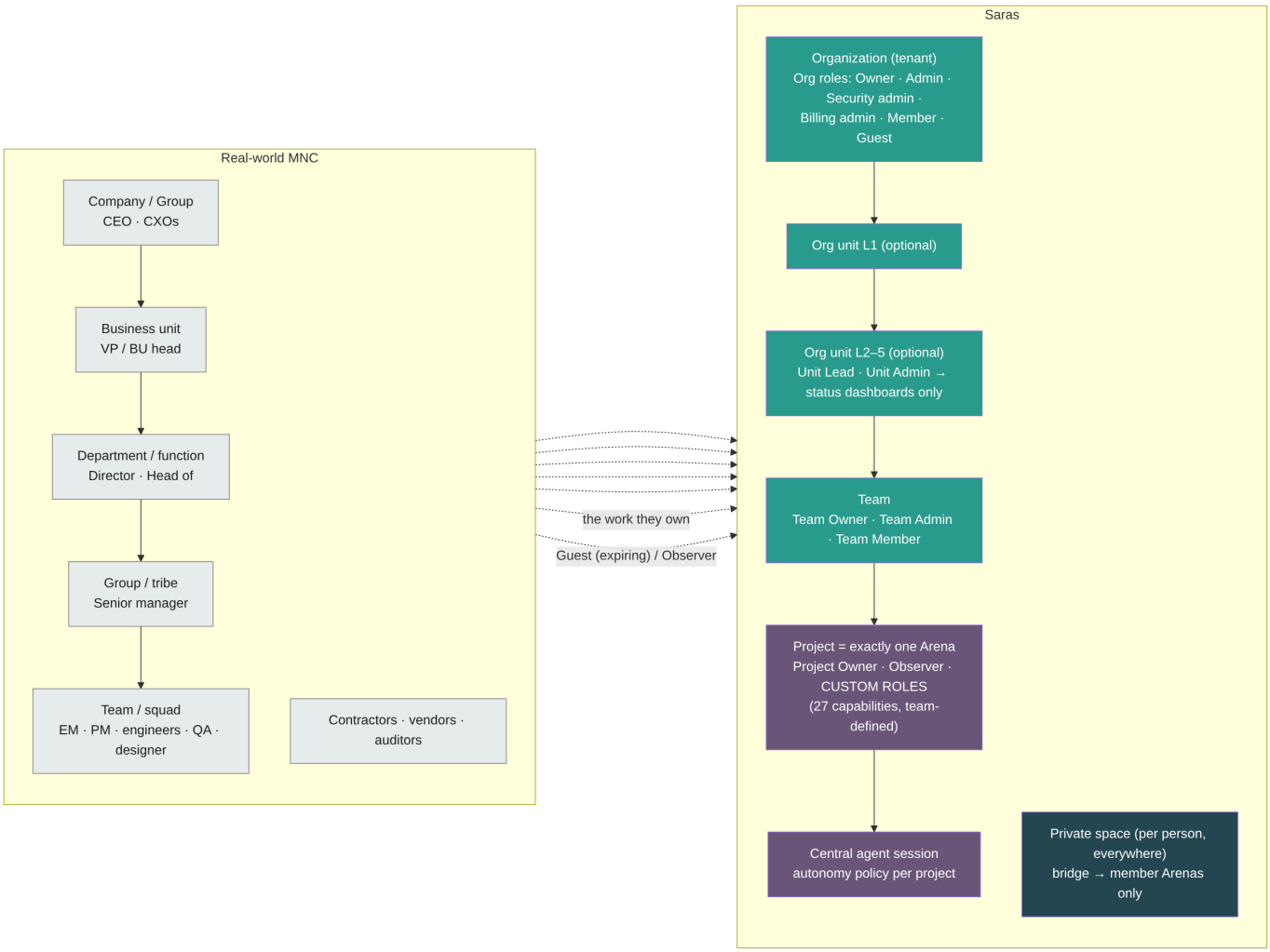
3.1 Design stance

Saras is **controlled by the teams that use it**. The organization provides identity, policy and oversight. Teams decide how they work: how many people they have, which roles exist, who approves what, and how much the AI is allowed to do.

Three rules follow from that stance and drive every permission decision:

1. **Permissions follow responsibility on the work, not seniority in the company.** A Director is not automatically an approver on a project. A junior engineer can be the approver for BUILD if the team says so. Job level is profile metadata. It never grants access.
2. **Oversight is not access.** People higher in the hierarchy (unit leads, org admins) see *status* (stages, progress, blockers, gate wait times) for their part of the organization. They see *content* (Arena messages, artifacts, tasks) only if they are members of that project, at least as Observers.
3. **AI never exceeds the humans it works for.** An agent's effective permission is the intersection of: what the project lets the agent do, what the human it acts for may do, and platform invariants (humans approve gates, push from Private, and manage membership).

3.2 How an MNC is organized, and how Saras maps it



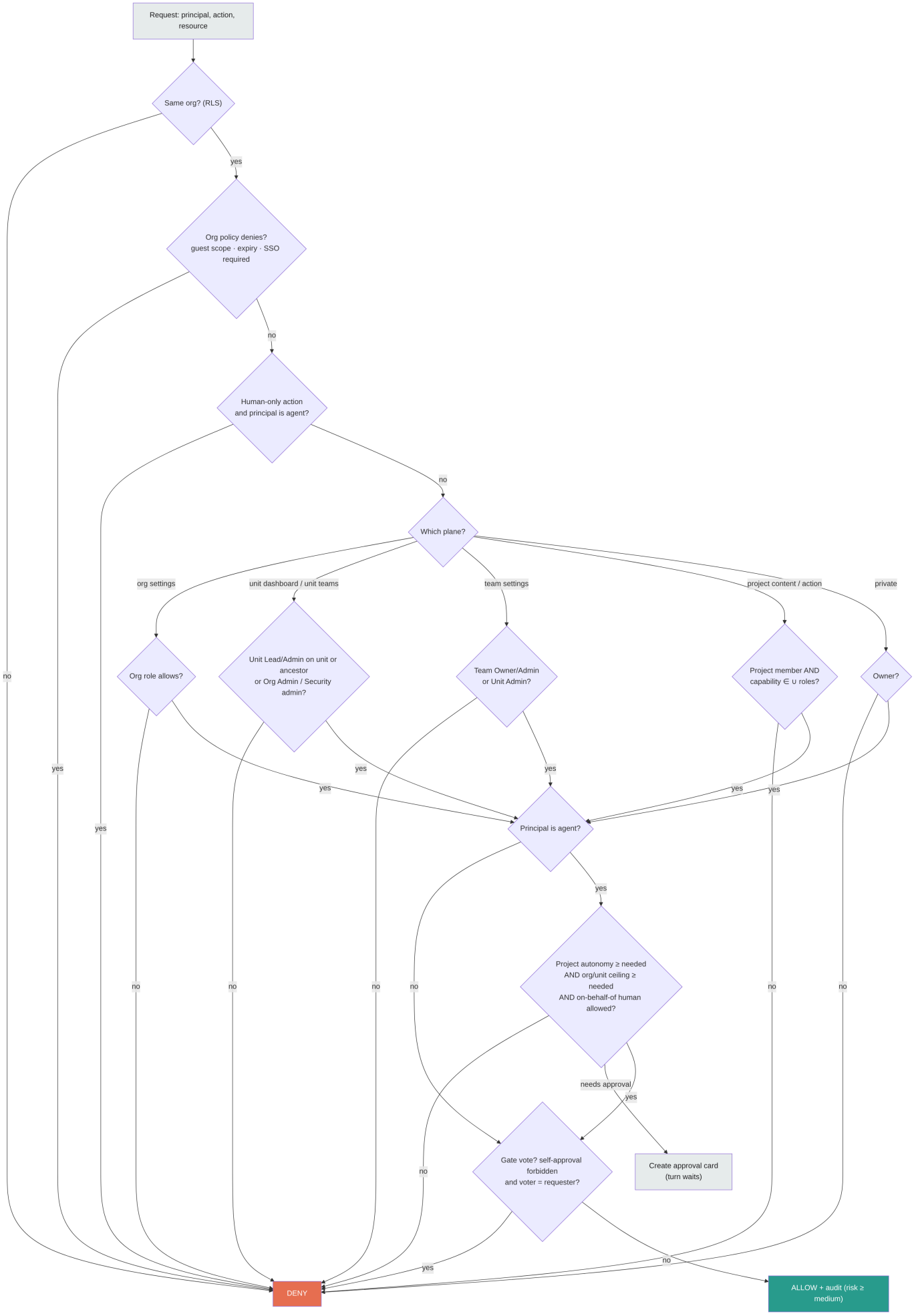
MNC hierarchy mapping

Real-world MNC layer	Typical people	Saras construct	What it is used for
Group / company (legal entity, region)	CEO, CXOs	**Organization** (tenant)	Identity, SSO/SCIM, org policies, billing, audit, org-wide knowledge for the central agent
Business unit / division	VP, BU head	**Org unit** (level 1)	Portfolio dashboard scope, policy overrides (allowed models, cross-unit invites), team grouping
Department / function (Engineering, Product, Design, SRE, Security)	Director, Head of	**Org unit** (level 2–3)	Same as above, narrower scope
Group / tribe	Senior manager	**Org unit** (level 3–5, optional)	Same
Team / squad (5–12 people, cross-functional)	Engineering manager, PM, engineers, QA, designer	**Team**	Self-governed group that owns projects. Team owners and admins manage it
Product / initiative / program	The people doing the work, plus stakeholders	**Project** with exactly one **Arena**	Collaboration, SDLC loop, artifacts, AI teammate. All content lives here
Cross-team program	Program manager + members of several teams	**Project** owned by one team, with members invited from other teams or units	Matrix collaboration without restructuring teams
Contractors, vendors, auditors	External people	**Guest** org members (time-bound) and/or project **Observers**	Least-privilege, expiring access to specific projects only
Individual	Everyone	**Private space** (global, owner-only)	Personal AI work. Crosses into Arenas only through the bridge

Org units are **optional**. A 20-person startup uses Organization → Teams → Projects. An MNC can model up to **5 unit levels** with its own level names (e.g. “Business unit → Department → Group”). Units are synced from the IdP via SCIM groups or edited by org admins.

Matrix organizations. People have one *home unit* (from HR/IdP) but can be members of any number of teams and projects. Project membership never depends on reporting lines. This supports functional reporting (e.g. QA department) alongside product squads.

3.3 Four permission planes


Request: principal, action, resource
Same org? (RLS)
yes
Org policy denies? guest scope · expiry · SSO required
no
Human-only action and principal is agent?
no
Which plane?
org settings
unit dashboard / unit teams
team settings
project content / action
private
Org role allows?
Unit Lead/Admin on unit or ancestor or Org Admin / Security admin?
Team Owner/Admin or Unit Admin?
Project member AND capability ∈ ∪ roles?
Owner?
Principal is agent?
yes
Project autonomy ≥ needed AND org/unit ceiling ≥ needed AND on-behalf-of human allowed?
needs approval
Gate vote? self-approval forbidden and voter = requester?
Create approval card (turn waits)
DENY
ALLOW + audit (risk ≥ medium)

Permission planes

Plane 1 — Organization roles (system-defined, fixed)

Org role	Who in an MNC	Can	Cannot
Org Owner (≥1)	Founder, CIO, platform owner	Everything an Org Admin can, plus: transfer ownership, delete the org, manage billing owners	—
Org Admin	IT / engineering-productivity / PMO admin	Members, invites, guests, org units, unit role assignments, SSO/SCIM, org policies, integrations, org knowledge, full-org dashboard (metadata)	Read Arena content of projects they are not members of; act inside a team's project settings
Security & Compliance Admin	CISO office, GRC	Audit log + export, retention settings, security policies (SSO required, guest expiry, allowed model classes, data region), full-org dashboard (metadata)	Manage members or teams; read Arena content
Billing Admin	Finance	Plan, invoices, seat counts, agent-usage budgets	Anything else
Member (default)	Every employee	Use Private space; create teams (if policy allows); join projects by invite or access request; see the org directory	Org settings
Guest	Contractors, vendors, external auditors	Only the projects they are explicitly added to; Private space	See the org directory beyond co-members, create teams, be added as project owner. Access **expires** (default 90 days, policy-controlled)

A user holds exactly one org role. Org Owner ⊃ Org Admin. Security and Billing admin are separate (separation of duties).

Plane 2 — Unit roles (scoped to a subtree of org units)

Unit role	Who	Can (within the unit and its descendants)
Unit Lead	VP, Director, Head of	Portfolio dashboard for the subtree (projects, stages, progress, blockers, gate wait times, agent usage); approve team creation in the unit (if the org requires it); assign Unit Admins
Unit Admin	Ops lead, PMO, chief of staff	Create teams in the unit, move teams between child units, manage team ownership when an owner leaves, view the subtree dashboard

Unit roles **never** grant Arena content access. A Unit Lead who needs content asks to join a project (as Observer or a working role). The project owner decides.

Plane 3 — Team roles (system-defined, simple)

Team role	Can
Team Owner (≥1)	Everything a Team Admin can + delete/archive the team, transfer ownership
Team Admin	Add/remove team members; create projects; set team defaults (preferred role templates, default gate policies, default agent autonomy)
Team Member	Be added to the team's projects; create projects if the team allows it (default: on)

Team size is unlimited by design. Plans may cap seats, not team structure.

Plane 4 — Project roles (the USP: team-defined)

Every project has two **system roles** plus any number of **custom roles**:

Role	Kind	Capabilities
Project Owner	System, ≥1 required, not deletable	All project capabilities

Role	Kind	Capabilities
Observer	System, optional	`arena.read`, `artifact.export`. For stakeholders, execs, auditors. Cannot post, push, approve or invoke agents
Custom roles	Created by the team (from templates or blank)	Any subset of the capability catalogue (§3.4)

A member can hold several roles. Their capabilities are the union of those roles.

Membership size is fully flexible. A project can have 1 person or 200. The plan's soft limit is 200 members per project in MVP, configurable per org. Stage gates require approvers only for stages the team enables.

3.4 Project capability catalogue (enforced by the system)

Group	Capability	Meaning	Risk
Arena	`arena.read`	Read messages, artifacts, tasks, loop state	low
	`arena.post`	Post, reply, tag people	low
	`arena.tag_roles`	Tag a whole role (`@Backend dev`), which notifies every holder	low
AI	`agent.invoke`	Ask `@central` in the Arena	low
	`agent.assign_task`	Assign a task to the AI teammate	medium
	`agent.configure`	Change the project's agent autonomy policy	high
	`routine.manage`	Create, edit, pause and run routines	medium
Artifacts	`artifact.push`	Push from Private into this Arena	low
	`artifact.edit`	Create new versions of project artifacts	low
	`artifact.export`	Download artifacts / progress.md	low
Tasks	`task.create`	Create tasks	low
	`task.assign`	Assign or reassign tasks to people	low
	`task.delegate`	Delegate to a person or a whole role (sends a delegation notification)	low
	`task.verify`	Mark acceptance criteria verified and close tasks	medium
Loop	`stage.request_gate`	Ask approvers to move the cycle forward	low
	`stage.approve.PLAN` … `stage.approve.DEPLOY`	Vote on that stage's gate (5 capabilities)	medium / high (DEPLOY)
	`cycle.open`	Open the next cycle from MAINTAIN	medium
People	`member.invite`	Add members (and Observers)	high
	`member.remove`	Remove members	high
	`role.manage`	Create, edit and delete custom roles; assign roles	high
	`access.approve`	Approve access requests	high
Project	`project.settings`	Name, goal, discoverability, gate policies, repo link	high
	`project.archive`	Archive, restore, decommission	high

There are 27 capabilities. The catalogue is fixed in code and seeded into `capabilities`. New capabilities ship with migrations and default to **off** for existing custom roles.

3.5 Role templates for MNC and startup teams

Templates are starting points. Every field is editable during setup and later by anyone with `role.manage`.

Template	Roles created (★ = Project Owner)	Default gate approvers
Enterprise product squad	★ Engineering Manager, Product Manager, Tech Lead, Software Engineer, QA Engineer, Designer, SRE, Security Champion, Stakeholder (Observer)	PLAN: PM · DESIGN: Tech Lead + PM (2) · BUILD: Tech Lead · TEST: QA Engineer · DEPLOY: Engineering Manager + SRE (2, no self-approval)

Template	Roles created (★ = Project Owner)	Default gate approvers
Platform / infrastructure	★ Platform Lead, Platform Engineer, SRE, Security Champion, Consumer team rep (Observer)	PLAN/DESIGN: Platform Lead · BUILD: Platform Engineer · TEST: SRE · DEPLOY: Platform Lead + Security Champion
Cross-team program	★ Program Manager, Workstream Lead, Architect, Engineer, Stakeholder (Observer)	PLAN: Program Manager · DESIGN: Architect · BUILD: Workstream Lead · TEST: Workstream Lead · DEPLOY: Program Manager
Frontend / Backend split	★ Lead, Frontend dev, Backend dev, Designer	DESIGN: Lead + Designer · BUILD: FE + BE · TEST/DEPLOY: Lead
Lean squad	★ Owner, Builder	PLAN/DESIGN/DEPLOY: Owner · BUILD/TEST: Builder
Research / prototype	★ Lead, Researcher	All gates: Lead; TEST: skip
Blank	★ Owner only	Must configure every enabled gate

Default capabilities per template role: the matrix in `contracts/role_templates_v2.json` (generated from the same source the API serves at `GET /v1/role-templates`).

3.6 Agent plane: what the AI teammate may do

The **central agent** (one session per project Arena) and each **personal agent** (Private) act inside permissions. Every project has an **agent autonomy policy**, set at setup and editable with `agent.configure`:

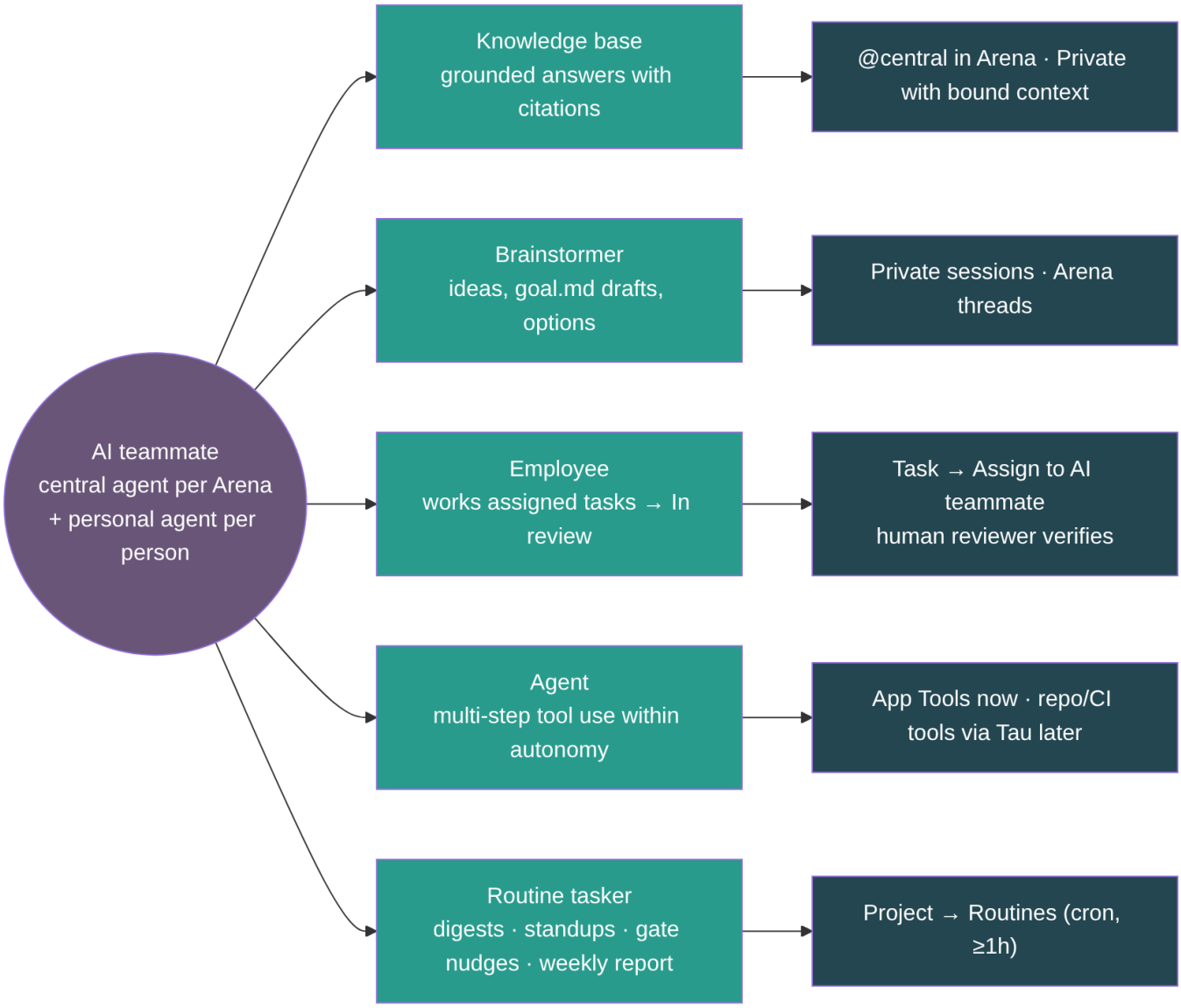
Capability class	Off	Suggest	Act with approval	Act
Answer and summarise (knowledge base, brainstormer)				**default**
Post in the Arena (replies, digests, routine output)				**default**
Draft artifacts (new draft versions only)				**default**
Create / update tasks		**default**		
Request a stage gate			**default**	
Work on assigned tasks (AI employee)			**default** (output goes to review)	
Tag / notify people				**default**

- **Suggest** = the agent proposes; a human clicks to apply.
- **Act with approval** = the agent's action creates an approval card routed to holders of the relevant capability.
- **Act** = allowed directly, but only where the human the agent acts for could do the same.

Never allowed for any agent (platform invariants): - approve or vote on gates; - verify/close a task it worked on; - invite, remove or change roles of members; - push from someone's Private space; - change agent policy; - archive projects; - read projects outside its session scope.

Org ceiling: org policy `agent_max_autonomy` (Security admin) caps every project. For example, a regulated BU may set "Act with approval" as the maximum.

3.7 The five AI roles and where each lives



AI roles

AI role (product language)	What it does	Surface	Permission path
Knowledge base	Answers questions grounded in the Arena, artifacts, tasks and org knowledge, with citations	`@central` in Arena; Private with bound context	`agent.invoke`; retrieval limited to session scope
Brainstormer	Explores ideas, drafts goal.md / options, challenges assumptions	Private sessions (personal agent); Arena threads	Private: owner only; Arena: `agent.invoke`
Employee (AI teammate)	Takes an assigned task, produces drafts and updates, moves the task to *In review*; a human verifies	Task panel “Assign to AI teammate”	`agent.assign_task`, autonomy “Work on assigned tasks”
Agent	Uses tools (App Tools now, repo / CI tools via Tau later) to complete multi-step work	Arena and Private	Autonomy policy ∩ invariants
Routine tasker	Runs scheduled routines: daily digest, standup collection, gate nudges, weekly progress report, custom prompts	Project → Routines	`routine.manage`; output posts as the central agent

3.8 Permission evaluation (exact algorithm)

```
can(principal, action, resource):
  0. Tenant: resource.org_id == principal.org_id                      else DENY (also enforced by Postgres RLS)
  1. Org policy denies (guest restrictions, SSO required, expiry)     → DENY
  2. Platform invariant (human-only action and principal is agent)    → DENY
  3. Plane by resource:
     org settings/members/units/policies → org role table (§3.3 P1)
     unit dashboard/teams-in-unit        → unit role on unit or any ancestor (P2) OR org admin/security admin
     team settings/members               → team role (P3) OR unit admin of team's unit OR org admin (membership only)
     project content and actions         → project membership required, then capability ∈ ∪(member's roles) (P4)
     private resources                   → owner only (no override for any role)
  4. If principal is an agent:
       require human_on_behalf_of passes step 3 for the same action
       require project agent policy class ≥ needed autonomy (Suggest → draft only; Approval → create approval)
       require org agent_max_autonomy ≥ that autonomy
  5. Gate votes: if policy.forbid_self_approval and voter == requester → DENY
  6. ALLOW → audit if action.risk ≥ medium
```

Implementation: - `packages/authz` is a pure function library: (principal, action, resource, facts) → decision + reason. - **Facts** (memberships, roles, capabilities, policies) are loaded per request and cached in Redis keyed by `membership_version`. - The **database** adds RLS for tenant isolation, Private ownership and Arena-membership visibility. It also checks the `arena.post` capability on message insert (`schema_v2.sql`), so a missing app check cannot leak or write data.

3.9 Access lifecycle (joiner, mover, leaver, guest)

Event	Source	What Saras does
Joiner	SCIM user created / invite accepted	Org membership (role Member, home unit from IdP group), personal agent identity, Private space. No project access until added or approved
Mover (changes unit/team)	SCIM group change / admin	Home unit updated → dashboards follow. Team and project memberships are **not** removed automatically. Team owners get a "member moved units" notice to decide
Leaver	SCIM deprovision / admin offboard	Sessions and tokens revoked within 60 s. Removed from teams and projects. Owned projects without another owner are flagged to the team admin (Unit Admin fallback). **Private content is retained per policy (default 30 days), then deleted.** Never transferred without a two-admin export request (post-MVP)
Guest expiry	Scheduled job	Access suspended at `expires_at`. Project owners notified 7 days before
Access request	Member finds a discoverable project	Request with reason → holders of `access.approve` decide and pick role(s) → requester notified

Project **discoverability:** - `hidden` (invite only — default for Security-tagged projects); - `team` (visible to the owning team); - `org` (listed in the org directory, requestable).

3.10 Separation of duties and enterprise controls (MVP)

Control	Setting	Default
No self-approval of gates	Per stage (`forbid_self_approval`)	ON for DEPLOY, OFF otherwise
Minimum approvals	Per stage	1 (2 for DEPLOY in enterprise templates)
Distinct roles for approval	`min_distinct_roles` per stage	1
Guest expiry	Org policy	90 days
Cross-unit project membership	Org policy	Allowed
Member team creation	Org policy	Allowed (MNC preset: Unit Admins only)
Agent autonomy ceiling	Org policy	Act
SSO required	Org policy	Off for startups / On for MNC preset
Data region	Org setting at creation	ap-south-1 (MVP only region)

Org presets at onboarding: - **Startup** (flat, permissive). - **Enterprise** (units on, SSO required, team creation by Unit Admins, DEPLOY needs 2 approvals with no self-approval, guests expire in 30 days, agent ceiling "Act with approval").

Admins can change any setting afterwards.

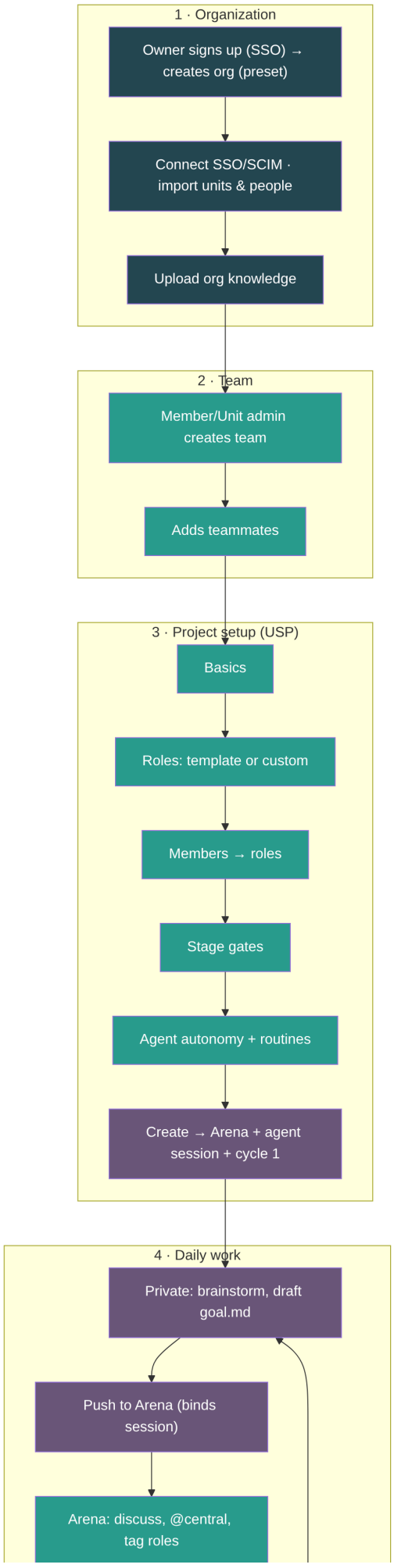
3.11 Worked examples

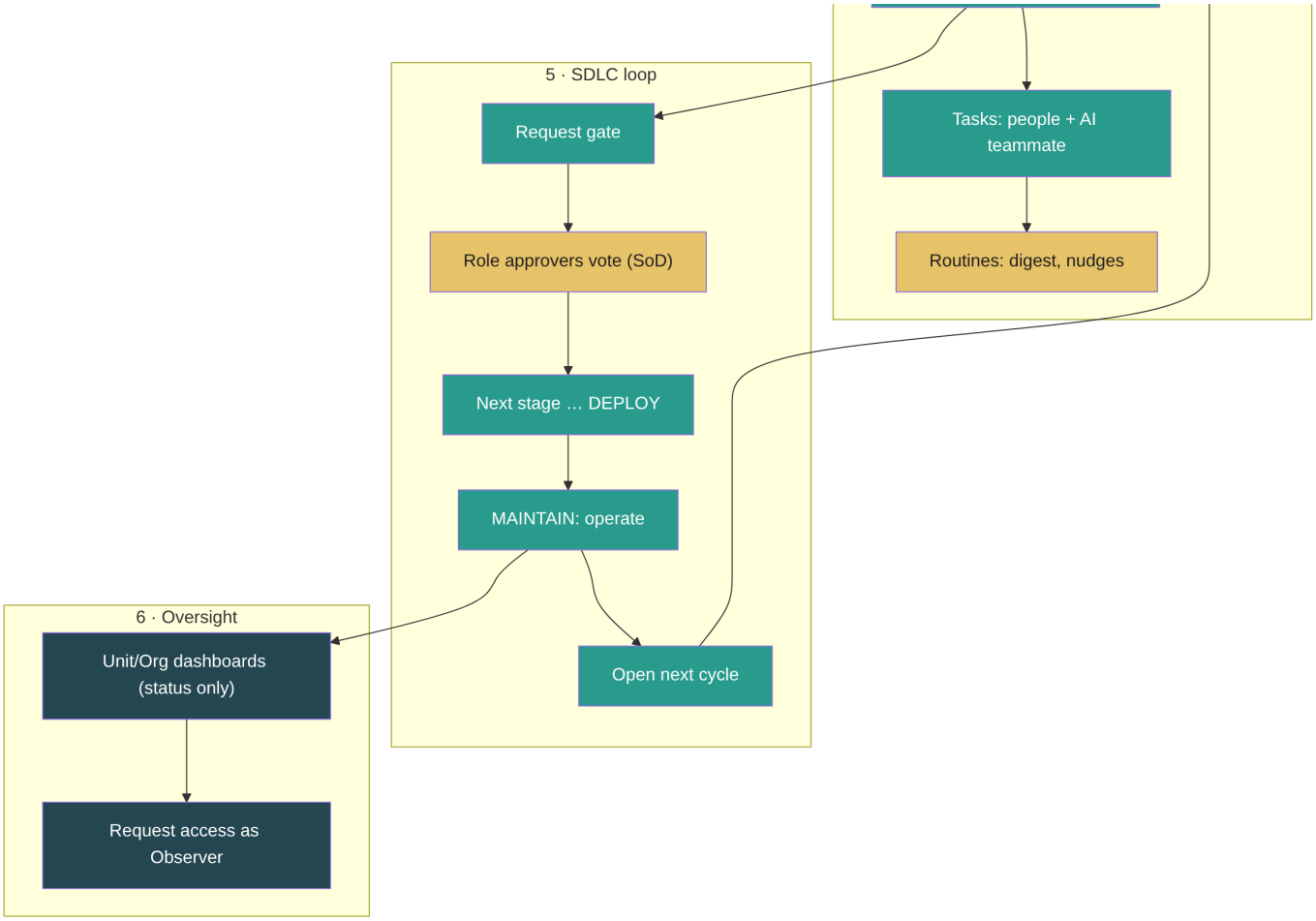
1. **VP of Payments (Unit Lead) asks: "How is the refunds revamp going?"** The VP sees the Payments subtree dashboard: REF is in Cycle 3 · BUILD, 54%, two tasks blocked, the DESIGN gate took 2 days. The VP has no Arena access. They click *Request access → Observer*. The project owner approves. The VP can now read the Arena, but cannot post or approve.
2. **Contractor joins for 6 weeks.** The Org Admin invites them as Guest (expires in 42 days). The team adds them to project PAY with the custom role "Frontend dev". They see only PAY. Access is suspended automatically on day 42.
3. **Junior engineer approves BUILD.** The team's custom role "Builder" holds `stage.approve.BUILD`. Seniority is irrelevant, as long as the role holder is not the requester when `forbid_self_approval` is on.
4. **AI teammate assigned "Draft API spec for refunds."** The agent acts on behalf of the assigner. It reads goal.md and Arena context, then writes `spec.md` as a draft and moves the task to *In review*. It cannot mark the task done. Someone with `task.verify` checks the acceptance criteria and closes it.
5. **Security admin sets agent ceiling to "Act with approval" for the Healthcare BU.** Every project in that subtree now gets approval cards for actions that were "Act", such as posting routine output.

04 — App Workflows & User Journeys

> Build Pack v2.0 · How the product behaves end to end: who does what, what the system does, which notifications fire. Screens referenced are on the design canvas “Saras App UI” (doc 09).

4.1 End-to-end flow


1 · Organization
Owner signs up (SSO) → creates org (preset)
Connect SSO/SCIM · import units & people
Upload org knowledge
2 · Team
Member/Unit admin creates team
Adds teammates
3 · Project setup (USP)
Basics
Roles: template or custom
Members → roles
Stage gates
Agent autonomy + routines
Create → Arena + agent session + cycle 1
4 · Daily work
Private: brainstorm, draft goal.md
Push to Arena (binds session)
Arena: discuss, @central, tag roles

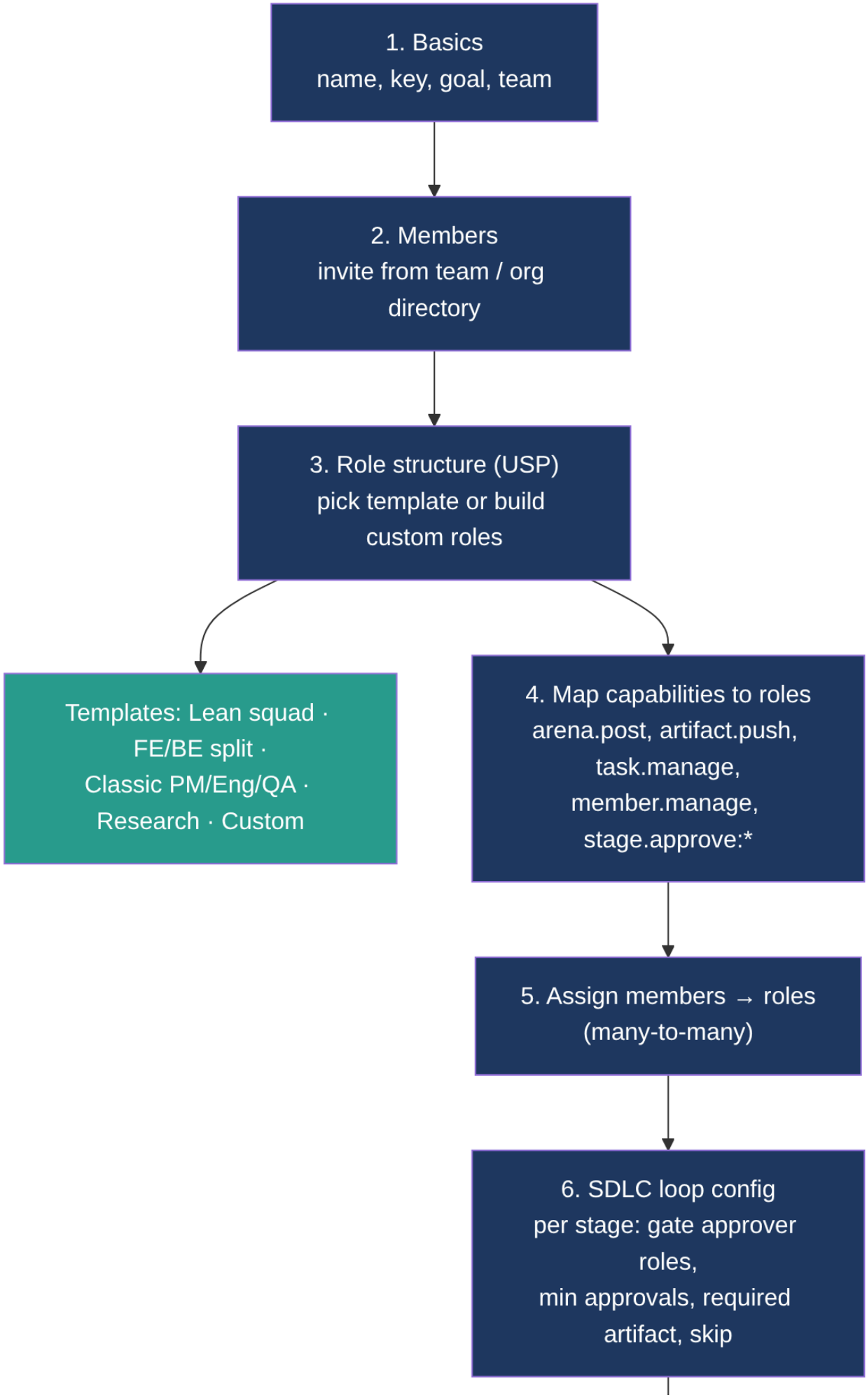


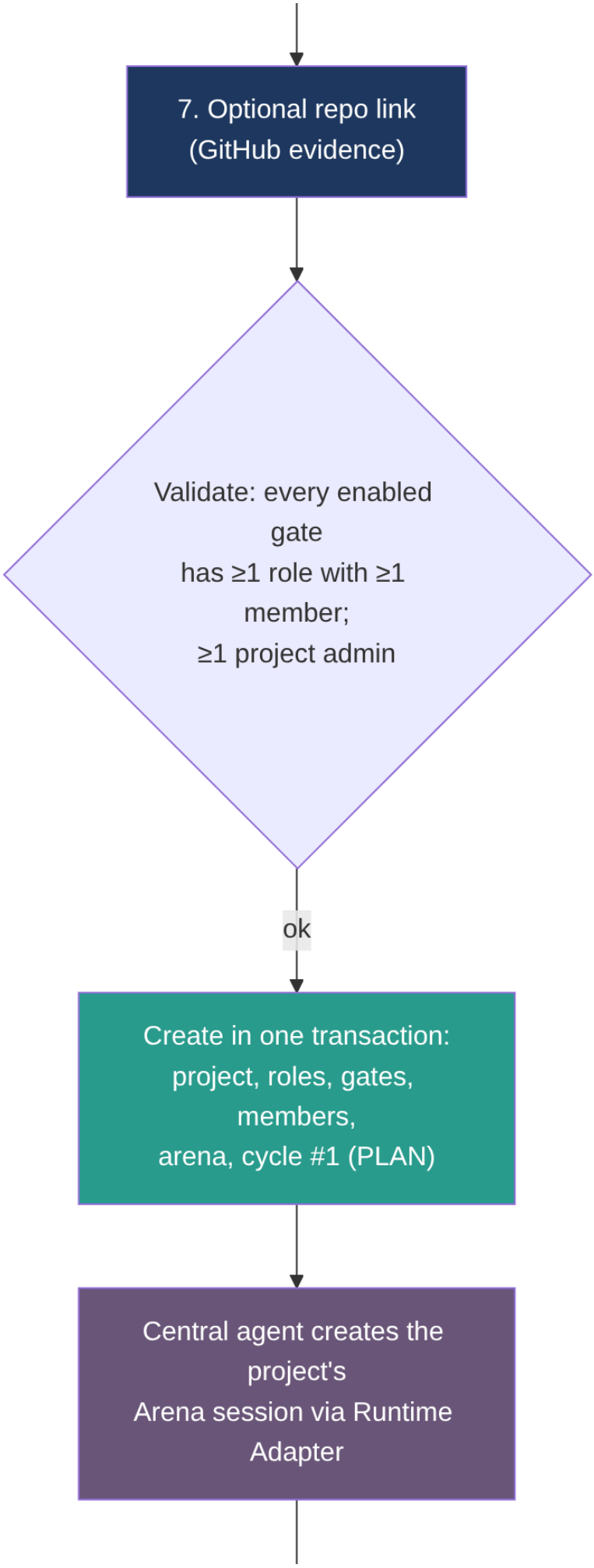
End-to-end journey

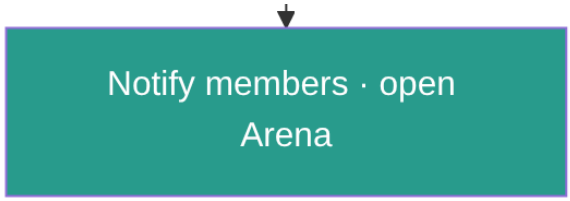
4.2 Journey A — Organization onboarding (Org Owner)

#	User does	System does	Screen
1	Signs in with Google / Microsoft / SSO	WorkOS AuthKit; creates `users` row	Login
2	Creates org: name, address, preset (Startup / Enterprise)	`organizations` + `org_policies` from preset; user = Org Owner; personal agent identity; Private space	Onboarding step 2
3	Uploads org knowledge (optional)	Files → S3 → indexing job → `knowledge_sources` ready; central agents use them	Onboarding step 3
4	Invites people or connects SSO/SCIM (Enterprise)	Invites emailed; or WorkOS portal link; SCIM creates members and home units	Onboarding step 4 / Org settings
5	(Enterprise) builds the unit tree, assigns Unit Leads/Admins	`org_units`, `unit_role_assignments`; audit	Org structure
6	Lands on the Org dashboard	Empty state: “Teams create projects. You’ll see their status here.”	Org dashboard

4.3 Journey B — Team and project setup (Team Owner → Project Owner)


1. Basics
name, key, goal, team
2. Members
invite from team / org directory
3. Role structure (USP)
pick template or build custom roles
Templates: Lean squad · FE/BE split · Classic PM/Eng/QA · Research · Custom
4. Map capabilities to roles
arena.post, artifact.push, task.manage, member.manage, stage.approve:*
5. Assign members → roles (many-to-many)
6. SDLC loop config
per stage: gate approver roles, min approvals, required artifact, skip


7. Optional repo link (GitHub evidence)
Validate: every enabled gate has ≥1 role with ≥1 member; ≥1 project admin
ok
Create in one transaction: project, roles, gates, members, arena, cycle #1 (PLAN)
Central agent creates the project's Arena session via Runtime Adapter



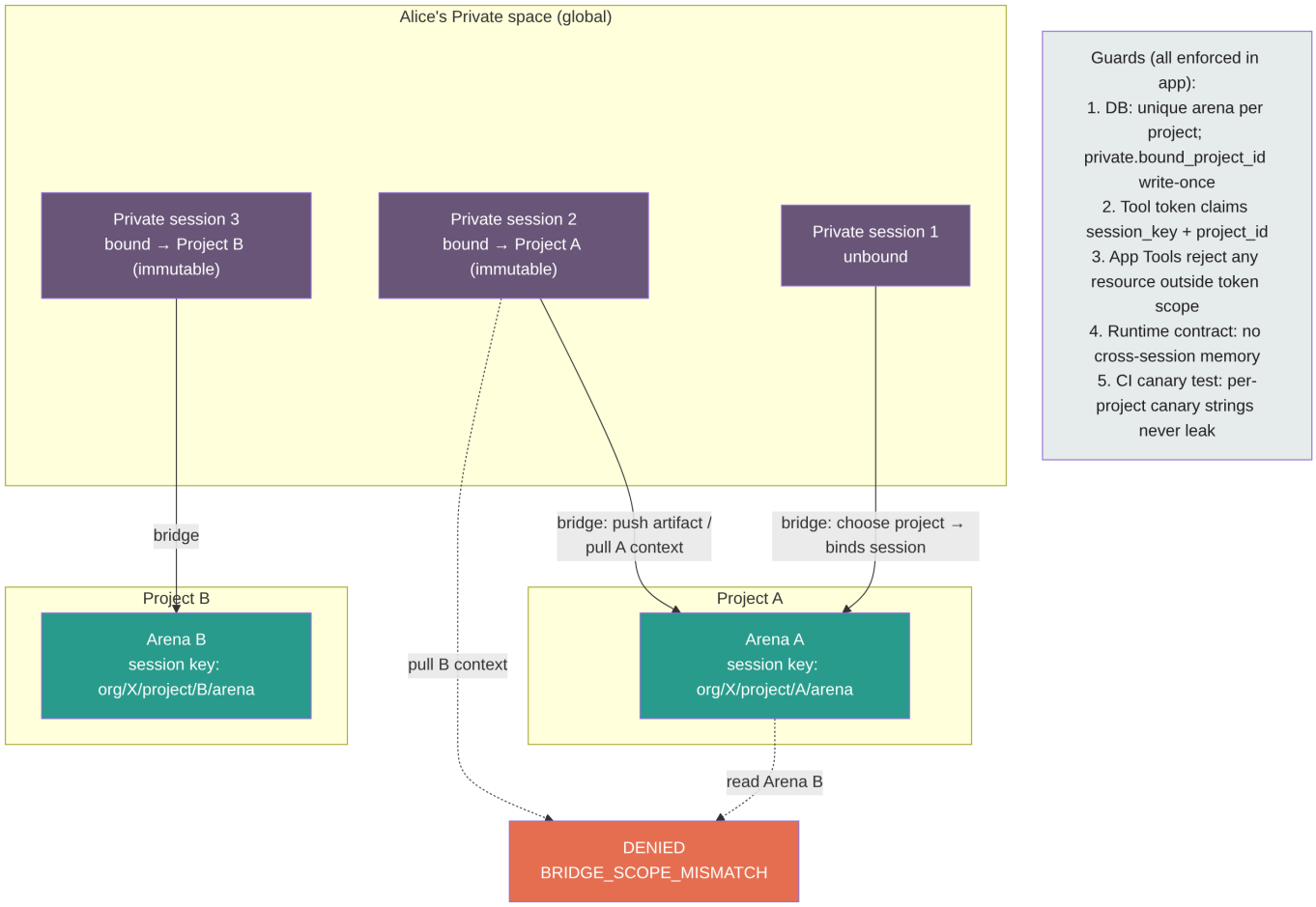
Project setup flow

#	User does	System does
1	Creates a team (if policy allows) and adds teammates	Team Owner role; members notified
2	Clicks **New project**: key, name, goal, discoverability	Project `status=setup`
3	Picks a role template (e.g. *Enterprise product squad*), renames roles, adds "Security champion", ticks capabilities	Roles kept in setup draft; Owner role always present
4	Assigns members to roles (many-to-many); adds a Stakeholder as Observer	Draft memberships
5	Configures gates: approver roles, min approvals, distinct roles, no-self-approval on DEPLOY, required artifacts; MAINTAIN cycle openers	Live validation (doc 02 F07 AC)
6	Sets agent autonomy (defaults from template, capped by org ceiling); enables starter routines (daily digest 09:30, gate nudge 24 h)	Draft policy and routines
7	Reviews and clicks **Create**	One transaction: roles, capabilities, members, gates, agent policy, routines, **Arena**, **cycle #1 at PLAN**. Then the runtime adapter provisions the **central agent session** (`org/{o}/project/{p}/arena`)
8	Arena opens	Central agent welcome message lists roles, gates and routines; members get a "you were added" notification

4.4 Journey C — Daily collaboration in the Arena

Action	Rule	Effect
Post a message	`arena.post`	Persisted → realtime to members → ambient batch to the agent (every 20 messages / 60 s)
Tag `@Karan`	`arena.post`	`tag` notification to Karan
Tag `@Backend dev`	`arena.tag_roles`	`tag` notification to every role holder
Ask `@central …`	`agent.invoke`	Turn (`mode=prompt`) → streamed answer with citations; steps visible in the drawer
Reply in thread	`arena.post`	`parent_id` set; thread counter updated
Attach a file	`arena.post`	Pre-signed upload → scan → attachment
Observer opens Arena	`arena.read` only	Read-only banner instead of composer

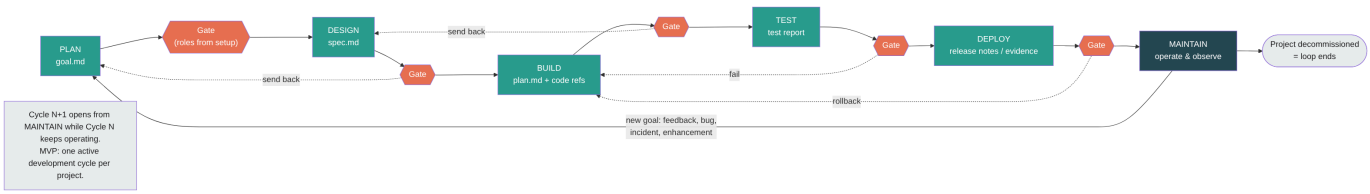
4.5 Journey D — Private → Arena (the bridge)



Session isolation

1. In **Private**, the user brainstorms with the personal agent. Optionally, they **bring context** from one project they're a member of. This binds the session to that project for good.
2. The agent drafts `goal.md` (a private artifact).
3. The user clicks **Push to Arena** and sees a preview: destination (locked if bound), new vs new version, diff, "request PLAN gate", "notify members", and what will be shared ("only goal.md v3").
4. On confirm, the system:
   - creates the project artifact version with lineage;
   - binds the session if it wasn't bound;
   - writes a `bridge_events` audit row;
   - optionally creates the gate approval;
   - sends `artifact_push` notifications.
5. Trying to pull context from another project in the same session → `BRIDGE_SCOPE_MISMATCH`, with the hint "start a new session".

4.6 Journey E — SDLC loop and gates

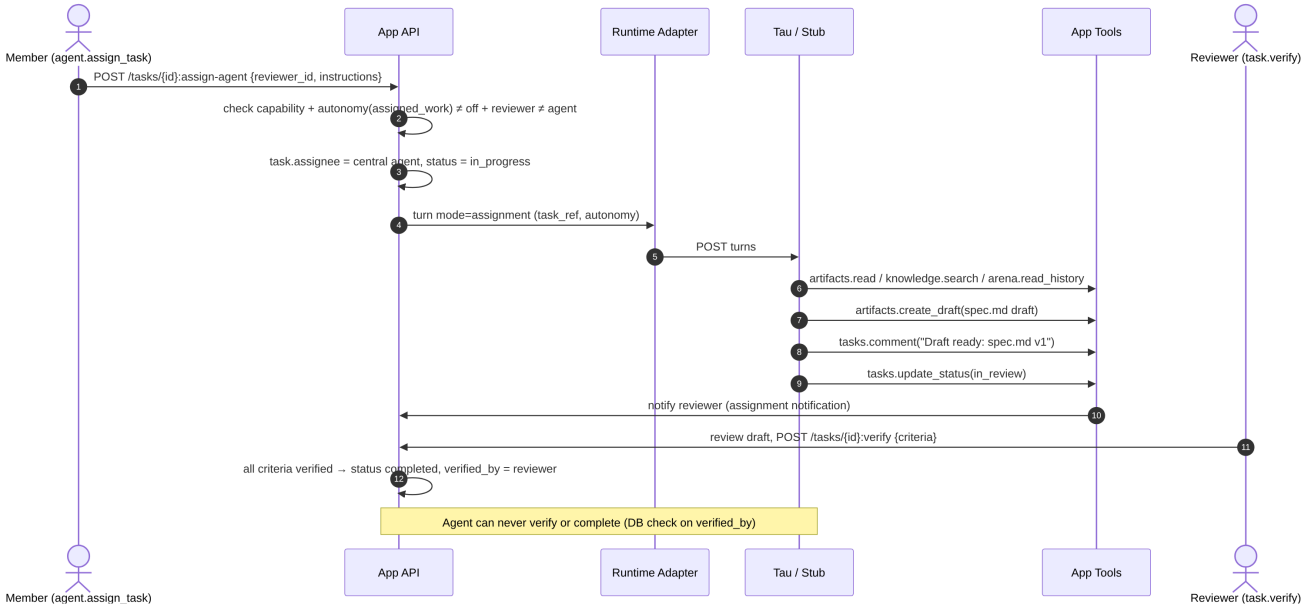


SDLC loop

Step	Who	Rule	Effect
Request gate	Holder of `stage.request_gate`	Required artifact must have an approved or in-review version	`approval_requests` routed to approver roles; `approval_request` notifications; card in Arena

Step	Who	Rule	Effect
Vote	Holder of `stage.approve.<STAGE>`	Human only; no self-approval if set; reject needs reason	`approval_votes`; when `min_approvals` and `min_distinct_roles` are met → advance (row lock)
Advance	System	—	`stage_transitions` (append-only); `stage_update` notifications; deterministic stage summary; `progress.md` regenerated; system turn to the agent; evidence commit (if repo linked)
Send back	Approver	Reason required	Back to the chosen earlier stage
MAINTAIN	—	Product is operating	Cycle status `operating`
Open next cycle	Holder of `cycle.open`	No other active cycle	New cycle at PLAN with reason (feedback / bug / incident / enhancement)
Decommission	Holder of `project.archive`	Reason	Loop ends; routines stop; Arena read-only

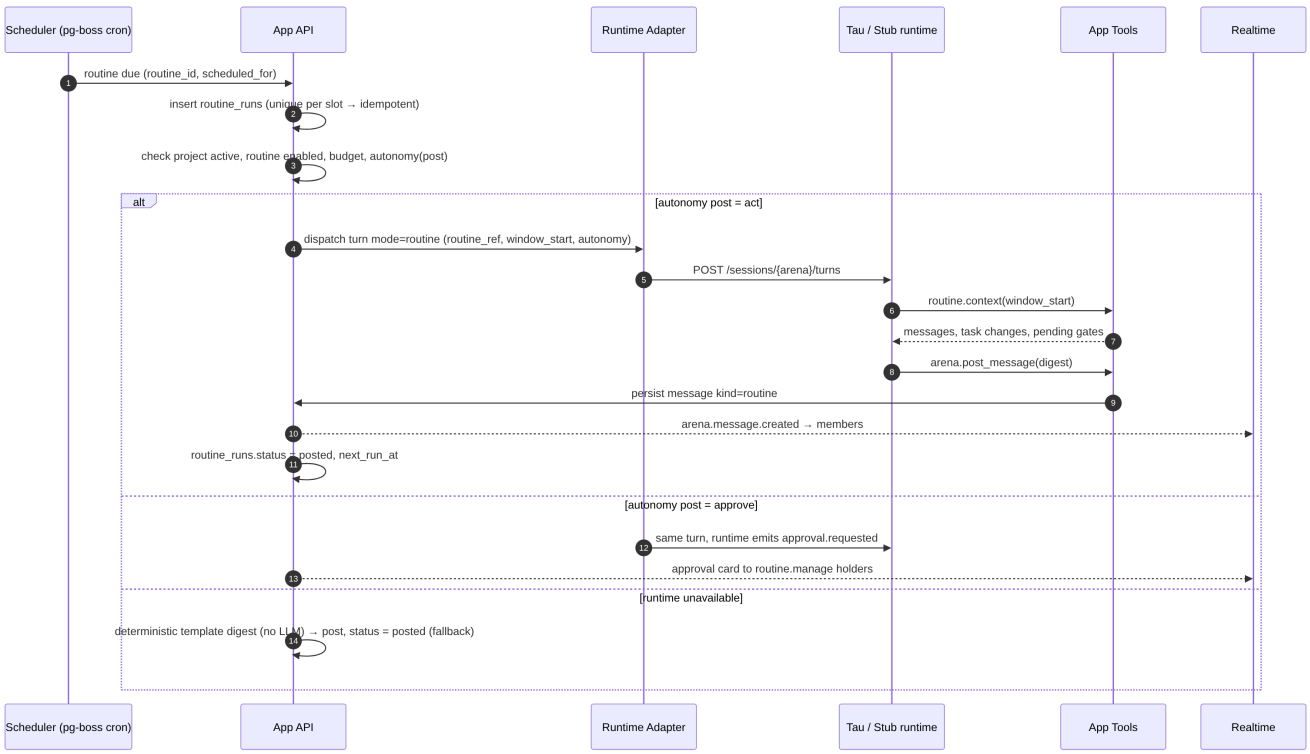
4.7 Journey F — AI teammate (AI as employee)



AI teammate

Rules: - Needs `agent.assign_task` and agent policy `assigned_work ≠ off`. - A human reviewer (not the assigner if self-approval rules require it) must be named. - The agent can create drafts, comment and move the task to *In review*. It can never verify or complete the task.

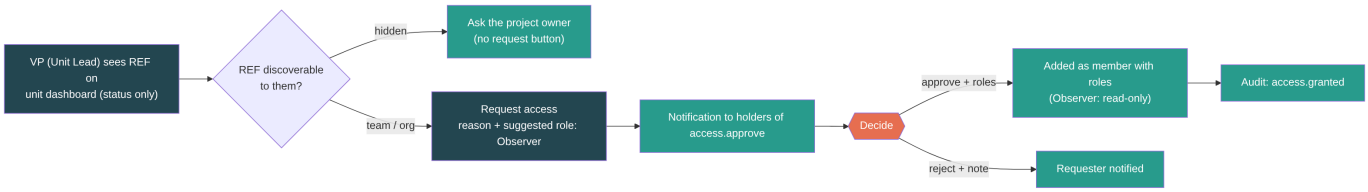
4.8 Journey G — Routines (AI as routine tasker)



Routine sequence

Template	Default schedule	What it posts
Daily digest	Weekdays 09:30 team timezone	Yesterday's decisions, artifact pushes, stage changes, blocked tasks, open gates
Standup collector	Weekdays 10:00	Asks members for updates in-thread; compiles a summary at 12:00
Gate nudge	Hourly check	Tags approvers of gates pending > N hours (default 24)
Weekly report	Fridays 16:00	`progress.md` summary + next 3 actions
Custom	Any cron ≥ 1 h	Result of the team's prompt over the last window

4.9 Journey H — Oversight and access (Unit Lead)



Access request

4.10 Journey I — People lifecycle

Event	Steps
Joiner	Invite or SCIM → org member → shows in directory → teams add them → projects add them with roles
Guest	Invite with expiry → added to specific projects only → warning 7 days before expiry → suspended
Mover	Home unit changes → dashboards follow → team owners notified to review memberships
Leaver	Offboard → sessions revoked → removed everywhere → owner-less projects flagged → Private deleted after retention

4.11 State machines (implementation reference)

Entity	States and transitions
Org membership	`invited → active → suspended → active` · `active → deprovisioned` (terminal)
Project	`setup → active ⇄ archived` · `active → decommissioned` (terminal)
Cycle	`active` (stage PLAN…DEPLOY) → `operating` (MAINTAIN) → `closed`; `active → abandoned`
Approval	`pending → approved / rejected / expired / cancelled`
Task	`backlog → ready → in_progress → in_review → completed` (verify only); `blocked` (reason) ⇄ previous; `cancelled`
Private session	`unbound → bound(project)` (write-once) → `archived`
Agent session	`provisioning → ready ⇄ degraded → archived / failed`
Turn	`queued → running → (awaiting_approval) → completed / failed / cancelled`
Routine run	`queued → running → posted / skipped / failed / awaiting_approval`
Access request	`pending → approved / rejected / cancelled`
Invite	`pending → accepted / revoked / expired`

4.12 Notification matrix

Type	Trigger	Recipients	Needs action
`artifact_push`	Push / new version in Arena	Project members (or `notify_role_ids`)	No
`stage_update`	Gate requested, passed, sent back; cycle opened	Project members	No
`approval_request`	Gate requested; agent action needs approval	Holders of the capability	Yes
`delegation`	Task delegated to person or role	Delegatee(s)	Yes
`assignment`	AI-teammate task ready for review	Reviewer	Yes
`tag`	`@person` / `@role`	Tagged users	No
`dm`	Direct message	Participants	No
`access_request`	Request to join a project	Holders of `access.approve`	Yes
`routine`	Routine failed or awaits approval	Holders of `routine.manage`	Sometimes
`system`	Guest expiry warning, member moved units, owner-less project	Owners / admins	Sometimes

05 — Architecture (HLD + LLD, consolidated)

> Build Pack v2.0 · Supersedes v1.0 docs 02–04 and v1.1 §3. Decisions and their alternatives: doc 13.

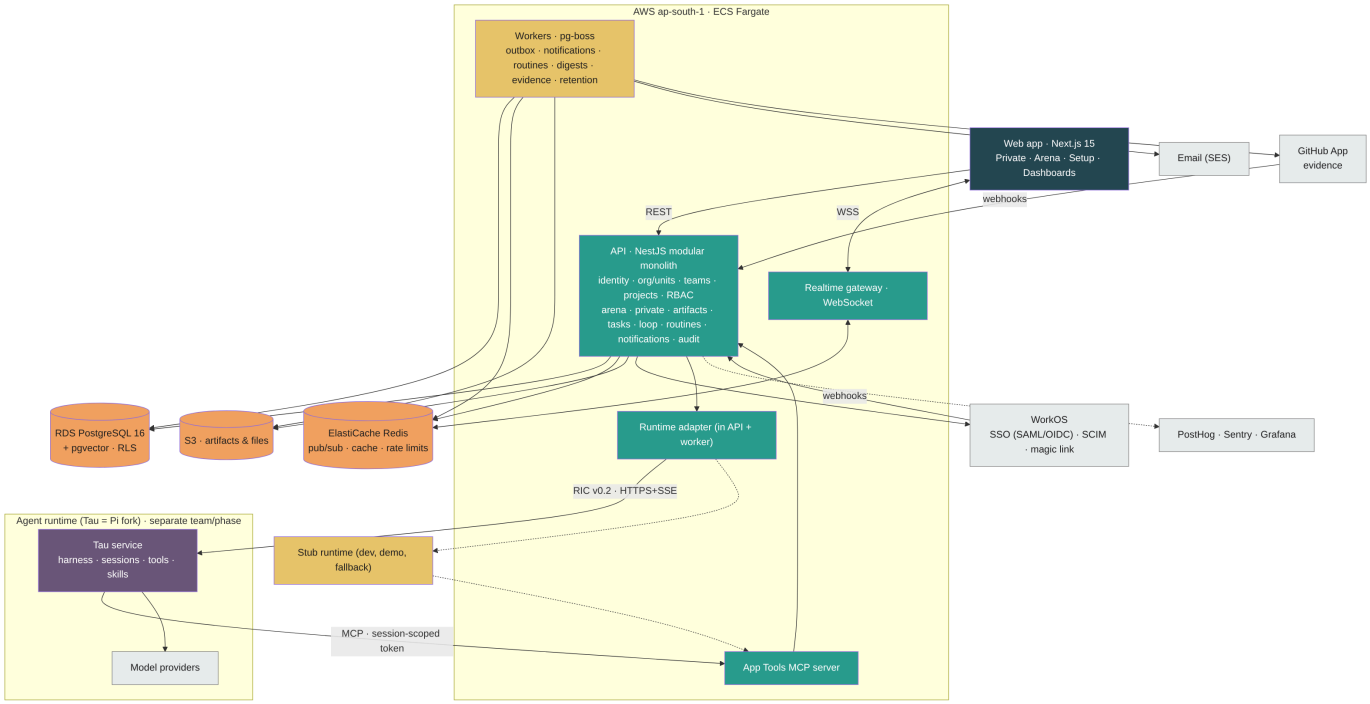
5.1 Shape of the system

Saras is two products joined by a contract:

1. **App platform (built from 16 Oct).** Identity, org hierarchy, RBAC, teams, projects, Arena, Private, bridge, artifacts, tasks, SDLC loop, routines, notifications, audit.
2. **Agent runtime "Tau" (separate team and phase).** A fork of the open-source Pi coding-agent harness. It holds the harness, sessions (tree + compaction), bundled tools and skills, model access and sandboxes.

They talk only through the **Runtime Integration Contract v0.2** (doc 08): - the app calls Tau over HTTPS + SSE; - Tau calls back into the app only through the **App Tools MCP server**, using session-scoped tokens.

Until Tau is ready, a **Stub Runtime** that implements the same contract powers every agent feature, including the YC demo.



Containers

5.2 Technology stack (final for MVP)

Layer	Choice	Notes
Web	**Next.js 15 (App Router), React 19, TypeScript, Tailwind, Radix/shadcn, TanStack Query, Zustand**	Design tokens from doc 09; Geist font
API	**NestJS 11 on Node 22 (TypeScript)**, modular monolith	Zod DTOs shared with web via `packages/contracts`
DB access	**Kysely** + plain SQL migrations (`graphile-migrate`)	Schema = `contracts/schema_v2.sql`
Database	**PostgreSQL 16** (AWS RDS, Multi-AZ) + **pgvector**	RLS for tenant, Private and Arena visibility
Jobs & schedules	**pg-boss** (Postgres-backed queues, cron)	Outbox relay, notifications, routines, digests, evidence, retention, guest expiry
Cache / realtime fan-out	**Redis 7** (ElastiCache)	Pub/sub for WebSocket fan-out, authz fact cache, rate limits, presence
Realtime	Own WebSocket gateway (`ws`)	5-minute tokens, topic authz, resume by seq

Layer	Choice	Notes
Files	**S3** (SSE-KMS), pre-signed uploads, ClamAV scan Lambda	Content-addressed artifact blobs
Auth	**WorkOS AuthKit + SSO + Directory Sync (SCIM)**	Google, Microsoft, magic link, SAML/OIDC, SCIM
Email	AWS SES	Invites, digests
Source control evidence	GitHub App	Commit approved stage artifacts, read PR/CI status
Hosting	**AWS ap-south-1, ECS Fargate**, ALB, CloudFront + WAF, Terraform	Simpler than Kubernetes for the MVP; Tau may run elsewhere
Observability	OpenTelemetry → Grafana Cloud; Sentry; uptime checks	doc 11
Product analytics & flags	PostHog (events + feature flags)	Ids only, no content
CI/CD	GitHub Actions → ECR → ECS (blue/green)	Preview environments per PR (P1)

5.3 Monorepo layout

```
saras/
├─ apps/
│  ├─ web/                Next.js app (Private, Arena, setup, loop, dashboards, inbox, settings)
│  ├─ api/                NestJS monolith (HTTP + adapter dispatch)
│  ├─ realtime/           WebSocket gateway
│  ├─ worker/             pg-boss consumers + cron (outbox, notifications, routines, evidence, retention)
│  ├─ app-tools/          MCP server exposed to the runtime
│  └─ stub-runtime/       RIC v0.2 implementation (scripted | thin-llm)
├─ packages/
│  ├─ contracts/          Zod schemas → OpenAPI v2, RIC types, event types, role_templates_v2.json
│  ├─ db/                 migrations (schema_v2), Kysely types, tenant-context helper
│  ├─ authz/              permission engine (doc 03 §3.8) — pure functions + exhaustive tests
│  ├─ domain/             state machines: cycle, gate, task, bind, routine, access request
│  ├─ runtime-client/     RIC client, SSE consumer, isolation guard
│  ├─ ui/                 design system components (tokens light/dark)
│  └─ observability/      OTel, logger with redaction
├─ infra/terraform/       VPC, ECS, RDS, ElastiCache, S3, SES, WAF, secrets
├─ tests/                 e2e (Playwright), canary isolation suite, RIC conformance kit
└─ docs/                  this Build Pack
```

5.4 API module map (bounded contexts)

Module	Owns tables	Key responsibilities
`identity`	users	Session, `/me`, WorkOS callback/webhooks
`org`	organizations, org_policies, org_memberships, invitations, knowledge_sources	Org, policies, directory, invites, offboarding
`units`	org_units, unit_role_assignments	Hierarchy, unit roles, dashboards scope
`teams`	teams, team_members	Teams and team roles
`projects`	projects, project_roles, role_capabilities, project_members, project_member_roles, stage_gate_policies, agent_policies, access_requests	Setup wizard, RBAC data, access
`arena`	arenas, messages, message_mentions	Arena stream, mentions, moderation
`private`	private_sessions, bridge_events	Private sessions, bind, pull context
`artifacts`	blobs, artifacts, artifact_versions, evidence, files	Versions, push, uploads, evidence
`tasks`	tasks	Tasks, delegation, AI teammate, verify
`loop`	sdlc_cycles, stage_transitions, approval_requests, approval_votes	Cycles, gates, votes, progress.md
`routines`	routines, routine_runs	Schedules and runs
`runtime`	agents, agent_sessions, runtime_turns, app_tool_calls	Adapter, turn ledger, usage
`messaging`	dm_threads, dm_participants (+messages)	DMs
`notify`	notifications, notification_prefs	Routing, inbox, email digest
`audit`	audit_events, outbox_events, processed_events	Audit chain, outbox

Module	Owns tables	Key responsibilities
`search`	(views + FTS indexes)	ACL-filtered search

Boundary rules: - A module touches only its own tables. - It reads other modules through their service interfaces. - Cross-module side effects go through outbox events (enforced by lint rules plus DB grants per schema in production).

5.5 Request lifecycle (LLD)

1. **Edge:** CloudFront/WAF → ALB → `api`.
2. **Middleware order:**
    1. request id;
    2. session/bearer auth → principal `{user_id, org_id}` (org from header `X-Saras-Org` or path);
    3. org membership status check (suspended/expired → 403 `MEMBERSHIP_INACTIVE`);
    4. rate limit (Redis token bucket);
    5. idempotency (`Idempotency-Key`, 24 h, Redis + response hash);
    6. Zod validation;
    7. **authz** (`packages/authz`, facts cached by `membership_version`);
    8. handler;
    9. problem+json mapper.
3. **DB transaction:**
    - `SET LOCAL app.org_id`, `app.user_id` (and `app.system_job` for workers);
    - domain writes, `audit_events` (risk ≥ medium) and `outbox_events` all in **one transaction**.
4. **After commit:**
    - the outbox relay (pg-boss job every 250 ms or LISTEN/NOTIFY) enqueues consumer jobs;
    - Redis publish for realtime.

5.6 Realtime

- `POST /v1/realtime/token` → 5-minute JWT.
- WebSocket `wss://rt.saras.app/v1/ws`.
- **Topics:**
    - `arena:{project}`;
    - `private:{session}` (owner only);
    - `dm:{thread}`;
    - `user:{id}` (notifications);
    - `turn:{turn}`.
- Subscribing is authorized with the same authz engine. On membership changes, the gateway force-unsubscribes.
- Frames carry a per-topic `seq`. On reconnect, the client sends `resume` and the gateway replays the last 500 frames from a Redis list, or tells the client to refetch over REST.
- REST is the only write path. WebSocket is for delivery only.

5.7 Runtime adapter (inside `api` + `worker`)

Concern	Design
Session provisioning	On setup complete: `POST /sessions` with `session_key`, profile `central_arena`, `context_seed` (org brief, project brief, roles, gates) and a tool token. Retries with backoff. Status in `agent_sessions`
Turn dispatch	`runtime_turns` row (id = idempotency key) → `POST /sessions/{id}/turns` with `mode`, input, `app_state` (cycle, stage), effective `autonomy`, `task_ref` / `routine_ref`
Event ingest	Worker consumes SSE: deltas → Redis publish (`turn:{id}`, `arena:{project}`). `message.completed` → persist agent message. `artifact.proposed` → draft artifact. `approval.requested` → approval card. `turn.completed` → usage
Isolation guard	Before every dispatch, recompute the expected `session_key` from the project or private session and compare. Mismatch → refuse, alert, audit
Ambient batching	Arena messages not addressed to `@central` are batched (20 messages / 60 s) as `reply_expected=false`
Fallback	If the runtime is unavailable: human chat continues, the agent is marked degraded, routines digest/nudge use deterministic templates, turns fail with `RUNTIME_UNAVAILABLE`

Concern	Design
Budgets	Per-org monthly budget (Billing admin). Turns refused with `BUDGET_EXCEEDED` at 100%, alerts at 80%

5.8 App Tools MCP server

- Streamable HTTP MCP endpoint.
- Bearer **tool token** carries `session_key`, `org_id`, `project_id`, principal (central agent / personal agent on behalf of a user), scopes and expiry (≤60 min).
- Every tool call runs this pipeline: resolve resources → reject anything outside `token.project_id` (`SCOPE_MISMATCH`) → authz (capability of the on-behalf-of user, or central-agent grant) → **autonomy check** (class from `app-tools_v0.2.mcp.json`; `approve` → creates an approval and returns `APPROVAL_REQUIRED`) → execute → log `app_tool_calls`.
- 20 tools; the human-only list is never exposed (doc 08).

5.9 Jobs and schedules (pg-boss)

Queue	Trigger	Job
`outbox.relay`	Continuous	Move outbox rows to consumer queues
`notify.fanout`	Domain events	Route notifications (doc 04 §4.12), apply prefs, publish realtime
`routine.tick`	Cron every minute	Find due routines → enqueue `routine.run` with unique `(routine, slot)`
`routine.run`	Tick	Dispatch `mode=routine` turn or deterministic fallback
`loop.after_transition`	Stage transition	Stage summary, `progress.md`, system turn, evidence commit
`evidence.git`	Approved stage artifact	GitHub App commit `.saras/cycles/{n}/{stage}.md`, retries
`members.guest_expiry`	Daily	Warn at 7 days, suspend at expiry
`members.offboard`	Offboarding	Revoke sessions, remove memberships, flag owner-less projects
`private.retention`	Daily	Delete Private content after retention for deprovisioned users
`knowledge.index`	Upload	Parse, chunk, embed (pgvector)
`approvals.expire`	Hourly	Expire stale approvals, notify requesters
`audit.archive`	Nightly	Export to S3 (Object Lock)

5.10 Caching

Data	Where	Invalidation
Authz facts (org role, unit roles, team roles, project capabilities)	Redis `authz:{org}:{user}:v{membership_version}`	Bump `membership_version` on any membership/role change
Project progress	Redis `progress:{project}:{tasks_version}`	`tasks_version` bump trigger
Role templates / capability catalogue	In-process	Deploy
Dashboard aggregates	Redis, 60 s	TTL

5.11 Errors and resilience

Class	Handling
Validation / authz / domain guard	4xx problem+json with stable code (doc 07 §7.4); no retry
Transient DB / Redis	Client retries with the same Idempotency-Key (exponential backoff + jitter, max 3)
Runtime unavailable	Degrade (above); banner in Arena; turns can be retried
GitHub / SES failures	Job retries up to 24 h; evidence shows `pending` / `failed`
Concurrency on gates	`SELECT … FOR UPDATE` on approval + cycle `version` check; double advance impossible

5.12 Deployment topology

Service	ECS tasks (min / max)	Scaling signal
web (Next.js)	2 / 8	CPU / requests
api	2 / 10	CPU / p95 latency
realtime	2 / 6	Connections per task (~5k)
worker	2 / 8	pg-boss queue depth
app-tools	2 / 6	Requests
stub-runtime (staging, demo, fallback)	1 / 2	—

Data: RDS PG16 `db.r7g.large` Multi-AZ + 1 read replica (search, dashboards) · ElastiCache Redis `cache.r7g.large` · S3 · SES.

Environments:

Environment	Notes
`local`	docker-compose: Postgres + pgvector, Redis, MinIO, Mailpit, stub runtime
`preview`	Per PR, P1
`staging`	Synthetic orgs + dogfood
`prod`	—

5.13 Capacity estimate (year 1)

Driver	Assumption	Load
Users	5,000 (DAU 50%)	—
Arena messages	40 / DAU / day	~100k/day, ~15/s peak
Agent turns	10 / DAU / day + routines	~30k/day
Storage	Messages ~1 KB; artifacts ~30 KB/version	< 200 GB year 1

The bottleneck is model latency and cost in the runtime, not the app. Back-pressure points are: turn queue, budgets, ambient batching.

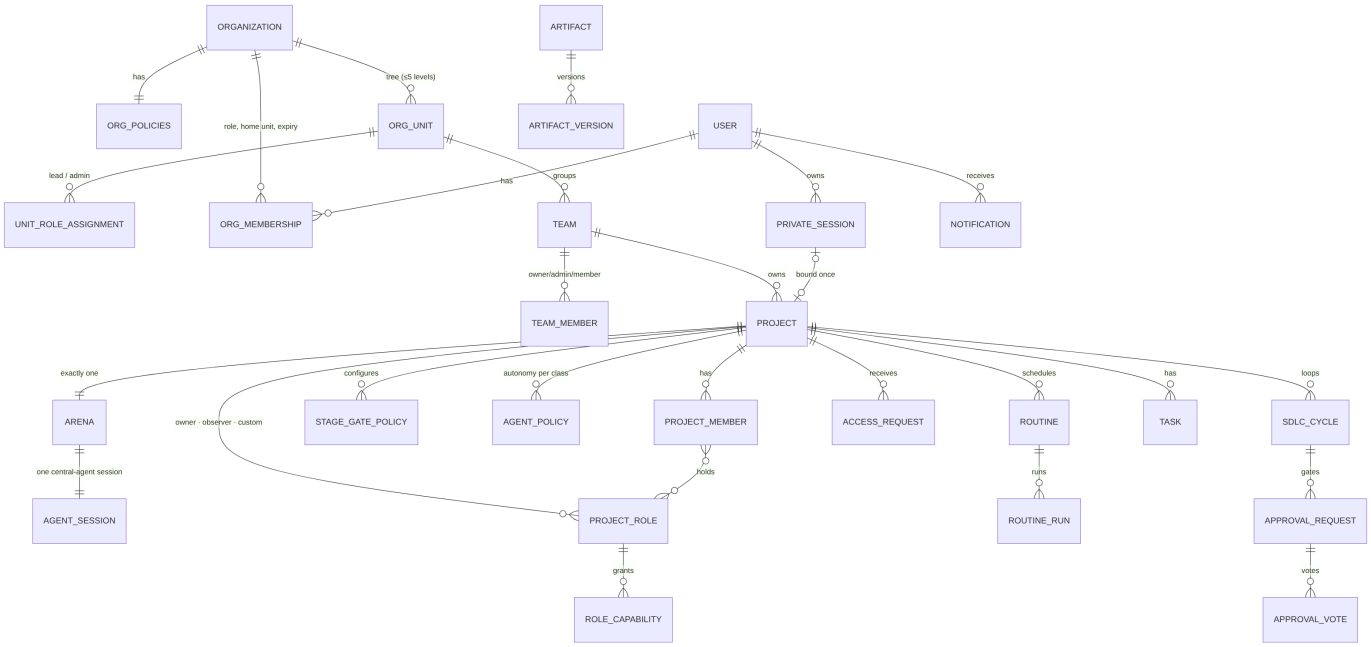
5.14 What we'd change at scale

Trigger	Change
> 5k orgs or DB CPU > 60% sustained	Split `arena` and `messaging` into their own service and database
> 50M vectors	Dedicated vector store
Many event consumers or replay needs	Kafka/Redpanda behind the outbox
Regulated enterprise deals	Single-tenant cells, BYO runtime / model keys, additional regions

06 — Data Model

> Build Pack v2.0 · Source of truth: `contracts/schema_v2.sql` (PostgreSQL 16 + pgvector). Supersedes `schema.sql` v1.0 and `schema_v1.1.sql`. Validated on PostgreSQL 16.15: 45 tables (+4 monthly message partitions), 2 views, RLS on 41 tables, 27 seeded capabilities.

6.1 Entity relationships



ERD v2

6.2 Table catalogue

Area	Tables	Notes
Organization	`organizations`, `org_policies`, `org_memberships`, `invitations`	Preset, WorkOS ids, region; one policy row per org; membership has role, status, home unit, level/title (metadata), reports_to, guest expiry, `membership_version`
Hierarchy	`org_units`, `unit_role_assignments`	Tree ≤5 levels, materialised `path` for subtree queries; Unit Lead / Unit Admin
Teams	`teams`, `team_members`	Unit link, team role owner/admin/member, team defaults
Identity & agents	`users`, `agents`	One personal agent per (org, user); one central agent per org (identity only; brains in Tau)
Projects & RBAC	`projects`, `capabilities`, `project_roles`, `role_capabilities`, `project_members`, `project_member_roles`, `stage_gate_policies`, `agent_policies`, `access_requests`	System roles (owner/observer) + custom roles; 27 capabilities; gate SoD controls; autonomy per class
Arena & sessions	`arenas`, `agent_sessions`, `messages` (partitioned), `message_mentions`	Exactly one Arena and one agent session per project; messages belong to exactly one of arena / private session / DM
Private & bridge	`private_sessions`, `bridge_events`	Write-once `bound_project_id`; every crossing audited
DMs	`dm_threads`, `dm_participants`	User-to-user only
SDLC loop	`sdlc_cycles`, `stage_transitions`, `approval_requests`, `approval_votes`, `evidence`	One active cycle; append-only transitions
Work	`tasks`	Agent assignee requires reviewer; completion requires verifier

Area	Tables	Notes
Artifacts	`blobs`, `artifacts`, `artifact_versions`	Content-addressed; one approved version
Routines	`routines`, `routine_runs`	Cron + timezone; unique run per slot
Runtime ledger	`runtime_turns`, `app_tool_calls`	Turns, usage, every tool call
Notifications	`notifications`, `notification_prefs`	10 types; dedupe key
Events & audit	`outbox_events`, `processed_events`, `audit_events`	Outbox, idempotent consumers, hash-chained audit
Views	`member_capabilities`, `org_dashboard_projects` (+ `project_progress` derivable)	`security_invoker = true` so the caller's RLS applies

6.3 Invariants enforced by the database

Invariant	Mechanism	Verified
Tenant isolation	RLS `tenant_isolation` (FORCE) on 41 tables via `app.org_id`	✓
Private is owner-only	RESTRICTIVE policies on `private_sessions`, private `artifacts`, private messages	✓
Arena content is member-only (org admins and unit leads are not implicit members)	RESTRICTIVE `message_visibility`	✓ Unit Lead (non-member) reads 0 messages
Observers cannot post	RESTRICTIVE INSERT `arena_post_requires_capability` (needs `arena.post`)	✓ insert rejected
One Arena per project; one agent session key	`arenas.project_id UNIQUE`, `agent_sessions.session_key UNIQUE`	✓
One Owner role and one Observer role per project	partial unique index on `(project_id, system_kind)`	✓
Owner role has all capabilities	`member_capabilities` view grants all to `system_kind='owner'`	✓ 27
Private session binds once	trigger `bound_project_write_once`	✓
One active cycle per project	partial unique index	✓
Approval gates name approver roles	CHECK on `stage_gate_policies`; `min_distinct_roles ≤ min_approvals`	✓
Guests must expire	CHECK on `org_memberships`	✓
AI-teammate tasks need a human reviewer; completion needs a verifier	CHECKs on `tasks`	✓
Audit, transitions and bridge events are append-only	`forbid_mutation` triggers	✓
One pending access request per user per project	partial unique index	—
Routine runs are idempotent	`UNIQUE (routine_id, scheduled_for)`	—

"✓" means the invariant was exercised in the v2 smoke test against PostgreSQL 16 (see `build/schema_v2_smoke.txt`).

6.4 Key rules not expressible in SQL (enforced in `packages/domain`)

- Every project keeps ≥1 member holding the Owner role (checked on remove/role change).
- An approval gate's roles must each have ≥1 member when the gate is used. Deleting a role or removing the last holder is blocked with `GATE_UNSTAFFED`.
- Agent autonomy ≤ org ceiling ≤ unit override (the effective policy is computed on read and sent to the runtime).
- Cross-unit membership allowed only if `allow_cross_unit_projects`.
- Guests cannot hold the Owner role, create teams, or see the directory beyond co-members.

6.5 Indexing and partitioning

Query	Index
Arena / Private / DM history	`messages (arena_id / private_session_id / dm_thread_id, created_at DESC)` partial indexes; monthly partitions (pg_partman creates 3 ahead)
Unit subtree dashboards	`org_units (org_id, path text_pattern_ops)`

Query	Index
My projects	`project_members (user_id, last_opened_at DESC)`
Board	`tasks (project_id, status)`
Due routines	`routines (next_run_at) WHERE enabled`
Inbox	`notifications (user_id, created_at DESC) WHERE resolved_at IS NULL`
Search	GIN on generated `tsvector` columns; pgvector HNSW on knowledge chunks (P1)

6.6 Retention

Data	Default	Configurable by
Arena messages, artifacts, transitions	Life of project + 1 year after decommission	Security admin (P1)
Private content of deprovisioned users	30 days, then deleted	`org_policies.private_retention_days_after_leave`
Audit events	400 days hot, 7 years in S3 Object Lock	Security admin
Runtime turns / tool calls	400 days	—
Outbox	7 days after publish	—

6.7 Migrations

- Tool: `graphile-migrate`.
- `schema_v2.sql` is the baseline migration `0001_baseline.sql`; later changes are incremental files.
- Rules:
  - expand → migrate → contract;
  - `CREATE INDEX CONCURRENTLY` on big tables;
  - each migration is reviewed with its RLS impact;
  - CI applies all migrations to a fresh database and runs the RLS / isolation suite.

07 — API Reference

> Build Pack v2.0 · Machine-readable: `contracts/openapi_v2.yaml` (OpenAPI 3.1 · 132 operations · 94 paths · 92 schemas · Redocly: 0 errors). Every operation carries `x-required-permission`; the table in §7.6 is generated from it.

7.1 Conventions

Topic	Rule
Base URL	`https://api.saras.app/v1` (staging: `api.staging.saras.app/v1`)
Auth	Web: `saras_session` cookie (HttpOnly) + CSRF header on mutations. Integrations/CI: Bearer token
Org context	Org-scoped paths include `{org_id}`. Other calls resolve the org from the resource; multi-org users send `X-Saras-Org`
IDs	UUIDv7
Pagination	`limit` (≤100) + opaque `cursor`; responses carry `next_cursor`
Idempotency	`Idempotency-Key` **required** on all creating and state-transition POSTs (24 h replay window)
Concurrency	`ETag` / `If-Match` on versioned resources (projects, tasks, artifacts) → `412` on mismatch
Errors	RFC 7807 `application/problem+json` with stable `code` and, on 403, `required_permission`
Visibility	Private resources and projects you can't read return `404`, not `403`
Content boundaries	Org and unit dashboards never return message, artifact or task content
Versioning	`/v1`; additive changes only; breaking change → `/v2` with 6-month overlap
Rate limits	600 req/min/user · 6,000 req/min/org · agent turns 30/min/user · headers `RateLimit-*`

7.2 Resource map

Area	Base paths
Identity	`/me`, `/auth/*`
Organization	`/orgs`, `/orgs/{org_id}/(policies / members / invites / sso / knowledge / usage / dashboard / audit-events)`
Hierarchy	`/orgs/{org_id}/units`, `/units/{unit_id}`, `/units/{unit_id}/(roles / dashboard)`
Teams	`/teams`, `/teams/{team_id}/members`
Projects & setup	`/capabilities`, `/role-templates`, `/projects`, `/projects/{id}/setup/*`, `/projects/{id}:archive / :restore / :decommission`
Roles & access	`/projects/{id}/(roles / members / access-requests / gates / agent-policy)`, `/access-requests/{id}:decide`
Arena	`/projects/{id}/arena`, `/projects/{id}/arena/messages`, `/messages/{id}`
Routines	`/projects/{id}/routines`, `/routines/{id}`
Private & bridge	`/private/sessions`, `/private/sessions/{id}/(messages / bridge/pull-context)`, `:bind`
Artifacts	`/artifacts`, `/artifacts/{id}/(versions / push-preview / push)`, `/files`
Tasks	`/projects/{id}/tasks`, `/tasks/{id}`, `:delegate`, `:assign-agent`, `:verify`
SDLC loop	`/projects/{id}/cycles`, `/cycles/{id}/(gate-requests / evidence)`, `:send-back`, `/approvals`, `/approvals/{id}/votes`
Messaging & notifications	`/dms`, `/notifications`, `/notification-prefs`, `/realtime/token`
Search, webhooks	`/search`, `/webhooks/workos`, `/webhooks/github`

7.3 Worked examples

Complete project setup (atomic):

```
POST /v1/projects/0193…/setup:complete
Idempotency-Key: 3f2c…
→ 200 { "id":"0193…", "key":"REF", "status":"active", "active_cycle":{"number":1,"stage":"PLAN"}, "my_roles":
["Engineering Manager"], … }
→ 422 { "code":"GATE_UNSTAFFED", "detail":"DEPLOY requires roles SRE, Engineering Manager; SRE has no members", … }
```

Observer tries to post:

```
POST /v1/projects/0193…/arena/messages
→ 403 { "code":"FORBIDDEN", "required_permission":"cap arena.post", "detail":"Your role in REF is Observer (read-only)."
}
```

Assign a task to the AI teammate:

```
POST /v1/tasks/0194…:assign-agent
Idempotency-Key: 9a1e…
{ "reviewer_id":"0191…", "instructions":"Draft the refunds API spec from goal.md v3", "model_class":"deep" }
→ 200 { "status":"in_progress", "assignee":{"type":"agent","id":"…central…"}, "reviewer_id":"0191…", … }
```

Vote with separation of duties:

```
POST /v1/approvals/0195…/votes   { "decision":"approve" }
→ 422 { "code":"SELF_APPROVAL_FORBIDDEN", "detail":"DEPLOY gate forbids approving your own request." }
```

7.4 Error codes

HTTP	code	When
400	`VALIDATION_FAILED`	Body / query invalid (`errors[]`)
401	`UNAUTHENTICATED`	No / expired session
403	`FORBIDDEN`	Missing permission (`required_permission` set)
403	`MEMBERSHIP_INACTIVE`	Suspended, expired guest, deprovisioned
403	`SSO_REQUIRED`	Org requires SSO
403	`HUMAN_ONLY_ACTION`	Agent attempted vote / verify / member or role change
404	`NOT_FOUND`	Missing or not visible
409	`ALREADY_EXISTS`, `CONFLICT`	Duplicate key / slug / name
409	`LAST_OWNER`	Removing the last Owner of a project or team
409	`ACTIVE_CYCLE_EXISTS`	Opening a cycle while one is active
409	`WRONG_STAGE`, `APPROVAL_CLOSED`	Loop guards
409	`BRIDGE_SCOPE_MISMATCH`, `ALREADY_BOUND`	Bridge guards
409	`IDEMPOTENCY_KEY_REUSED`	Same key, different body
410	`INVITE_EXPIRED`	Invite expired / revoked
412	`PRECONDITION_FAILED`	`If-Match` mismatch
422	`GATE_UNSTAFFED`, `DISTINCT_ROLES_UNSATISFIABLE`, `NO_OWNER`	Setup / role validation
422	`AUTONOMY_ABOVE_CEILING`	Agent policy above org/unit ceiling
422	`SELF_APPROVAL_FORBIDDEN`, `REASON_REQUIRED`, `REQUIRED_ARTIFACT_MISSING`	Gate rules
422	`ACCEPTANCE_CRITERIA_UNVERIFIED`, `REVIEWER_REQUIRED`	Task rules
422	`MEMBER_LIMIT_REACHED`, `CROSS_UNIT_NOT_ALLOWED`, `GUEST_RESTRICTED`	Membership rules
422	`CRON_TOO_FREQUENT`	Routine schedule < 1 h

HTTP	code	When
429	`RATE_LIMITED`, `BUDGET_EXCEEDED`	With `Retry-After`
503	`RUNTIME_UNAVAILABLE`	Agent features degraded

7.5 Domain events (outbox → pg-boss consumers)

Envelope: `{id, type, org_id, subject, actor{type,id,on_behalf_of}, time, trace_id, schema_version, data}`. Consumers are idempotent on `id`.

Event	Main consumers
`org.created`, `org.policies_changed`	audit, cache bust
`member.invited`, `member.joined`, `member.role_changed`, `member.moved_unit`, `member.offboarded`, `guest.expiring`, `guest.suspended`	notify, authz cache, offboarding job
`unit.created`, `unit.moved`, `unit.roles_changed`	dashboards cache, authz cache
`team.created`, `team.member_changed`	notify, authz cache
`project.setup_completed`, `project.archived`, `project.decommissioned`	runtime adapter (session), routines, notify
`project.role_changed`, `project.member_added`, `project.member_removed`	authz cache, tool-token rotation, notify
`access.requested`, `access.decided`	notify, audit
`arena.message_posted`, `mention.created`	realtime, notify, ambient batcher, search index
`turn.requested`, `turn.completed`, `turn.failed`	adapter, usage, realtime
`artifact.version_created`, `artifact.pushed`	notify, search index, evidence
`task.created`, `task.delegated`, `task.assigned_agent`, `task.in_review`, `task.verified`	notify, progress, adapter
`gate.requested`, `gate.voted`, `cycle.advanced`, `cycle.sent_back`, `cycle.opened`	notify, progress.md, evidence, system turn
`routine.due`, `routine.posted`, `routine.failed`	adapter, notify
`dm.message_posted`	realtime, notify

7.6 Endpoint catalogue with authorization rules (generated)

Tag	Method	Path	Summary	Required permission
Identity	GET	`/me`	Current user, org memberships, unit roles, personal agent	authenticated
Identity	PATCH	`/me`	Update display name / handle	authenticated
Identity	GET	`/auth/login`	Start login (redirects to WorkOS AuthKit: SSO, Google, Microsoft, magic link)	public
Identity	GET	`/auth/callback`	OAuth callback; sets session cookie	public
Identity	POST	`/auth/logout`	End session	authenticated
Organization	POST	`/orgs`	Create an organization (creator becomes Org Owner) with a preset	authenticated
Organization	GET	`/orgs/{org_id}`	Organization details	org member
Organization	PATCH	`/orgs/{org_id}`	Rename, unit level names	org:owner|admin
Organization	GET	`/orgs/{org_id}/policies`	Org policies	org:owner|admin|security_admin
Organization	PUT	`/orgs/{org_id}/policies`	Update org policies (security fields require security_admin or owner)	org:owner|admin|security_admin
Organization	GET	`/orgs/{org_id}/members`	Directory (guests see only co-members)	org member

Tag	Method	Path	Summary	Required permission
Organization	PATCH	`/orgs/{org_id}/members/{user_id}`	Change org role, home unit, title, level, guest expiry	org:owner|admin
Organization	DELETE	`/orgs/{org_id}/members/{user_id}`	Offboard: revoke sessions, remove from teams/projects, schedule Private retention	org:owner|admin
Organization	GET	`/orgs/{org_id}/invites`	Pending invites	org:owner|admin
Organization	POST	`/orgs/{org_id}/invites`	Invite people to org / team / project (guests need guest_access_until)	org:owner|admin OR team:owner|admin (team scope) OR cap member.invite (project scope)
Organization	DELETE	`/orgs/{org_id}/invites/{invite_id}`	Revoke invite	inviter OR org:owner|admin
Organization	POST	`/invites/{token}:accept`	Accept an invite	authenticated (email must match)
Organization	GET	`/orgs/{org_id}/sso`	SSO / SCIM status and WorkOS admin-portal link	org:owner|admin
Organization	POST	`/orgs/{org_id}/sso:portal-link`	Generate WorkOS admin-portal link to configure SAML/OIDC/SCIM	org:owner|admin
Units	GET	`/orgs/{org_id}/units`	Org unit tree	org member
Units	POST	`/orgs/{org_id}/units`	Create unit (max depth 5)	org:owner|admin
Units	PATCH	`/units/{unit_id}`	Rename / move unit / policy overrides	org:owner|admin
Units	DELETE	`/units/{unit_id}`	Delete an empty unit	org:owner|admin
Units	PUT	`/units/{unit_id}/roles`	Assign Unit Leads and Unit Admins	org:owner|admin OR unit_lead (admins only)
Units	GET	`/units/{unit_id}/dashboard`	Portfolio dashboard for the unit subtree (metadata only)	unit_lead|unit_admin on unit or ancestor OR org:owner|admin|security_admin
Organization	GET	`/orgs/{org_id}/dashboard`	Organization dashboard (metadata only)	org:owner|admin|security_admin
Organization	GET	`/orgs/{org_id}/knowledge`	Org knowledge sources used by the central agent	org member
Organization	POST	`/orgs/{org_id}/knowledge`	Add knowledge source (uploaded file) for indexing	org:owner|admin OR unit_admin (unit scope)
Organization	DELETE	`/knowledge/{knowledge_id}`	Remove knowledge source	org:owner|admin
Organization	GET	`/orgs/{org_id}/usage`	Agent usage and budget	org:owner|admin|billing_admin
Audit	GET	`/orgs/{org_id}/audit-events`	Audit log	org:owner|security_admin
Audit	POST	`/orgs/{org_id}/audit-exports`	Export audit log (CSV/JSON)	org:owner|security_admin
Teams	GET	`/teams`	Teams I belong to (filter=all for directory)	org member
Teams	POST	`/teams`	Create a team (creator = Team Owner); subject to org policy member_team_creation	per org policy
Teams	GET	`/teams/{team_id}`	Team details	org member (non-guest) OR team member
Teams	PATCH	`/teams/{team_id}`	Update team settings	team:owner|admin
Teams	DELETE	`/teams/{team_id}`	Archive team (projects must be archived first)	team:owner OR unit_admin

Tag	Method	Path	Summary	Required permission
Teams	GET	`/teams/{team_id}/members`	Team members	team member
Teams	POST	`/teams/{team_id}/members`	Add members	team:owner|admin
Teams	PATCH	`/teams/{team_id}/members/{user_id}`	Change team role	team:owner (owner/admin) \
Teams	DELETE	`/teams/{team_id}/members/{user_id}`	Remove member	team:owner|admin
Projects	GET	`/capabilities`	Capability catalogue (27)	authenticated
Projects	GET	`/role-templates`	Role templates with roles, capabilities, gates and agent policy	authenticated
Projects	GET	`/projects`	My projects or discoverable projects	org member
Projects	POST	`/projects`	Create project draft (status=setup)	team member if team allows, else team:owner|admin
Projects	GET	`/projects/{project_id}`	Project overview (members only; others get discoverable summary)	cap arena.read OR discoverable
Projects	PATCH	`/projects/{project_id}`	Update project	cap project.settings
Projects	PUT	`/projects/{project_id}/setup/roles`	Setup step: roles (template or custom)	project creator during setup
Projects	PUT	`/projects/{project_id}/setup/members`	Setup step: members and role assignment	project creator during setup
Projects	PUT	`/projects/{project_id}/setup/gates`	Setup step: stage gate policies	project creator during setup
Projects	PUT	`/projects/{project_id}/setup/agent`	Setup step: agent autonomy policy (capped by org ceiling)	project creator during setup
Projects	POST	`/projects/{project_id}/setup:validate`	Validate setup (≥1 owner, every approval gate staffed, distinct roles feasible)	project creator during setup
Projects	POST	`/projects/{project_id}/setup:complete`	Atomically create roles, members, gates, Arena, cycle #1 and provision the central agent session	project creator during setup
Projects	POST	`/projects/{project_id}:archive`	Archive (pauses loop and routines)	cap project.archive
Projects	POST	`/projects/{project_id}:restore`	Restore	cap project.archive
Projects	POST	`/projects/{project_id}:decommission`	Decommission: ends the SDLC loop permanently	cap project.archive
Roles & access	GET	`/projects/{project_id}/roles`	Project roles with capabilities	cap arena.read
Roles & access	POST	`/projects/{project_id}/roles`	Create custom role	cap role.manage
Roles & access	PATCH	`/projects/{project_id}/roles/{role_id}`	Rename / change capabilities (system roles: name only)	cap role.manage
Roles & access	DELETE	`/projects/{project_id}/roles/{role_id}`	Delete custom role (must not leave a gate unstaffed)	cap role.manage
Roles & access	GET	`/projects/{project_id}/members`	Members and their roles	cap arena.read
Roles & access	POST	`/projects/{project_id}/members`	Add members (respects member_limit, cross-unit policy, guest rules)	cap member.invite (+ role.manage to grant non-observer roles)
Roles & access	PATCH	`/projects/{project_id}/members/{user_id}`	Set a member's roles	cap role.manage

Tag	Method	Path	Summary	Required permission
Roles & access	DELETE	`/projects/{project_id}/members/{user_id}`	Remove member (cannot remove last owner)	cap member.remove OR self (leave)
Roles & access	POST	`/projects/{project_id}/access-requests`	Request access to a discoverable project	org member (non-guest), project discoverable to them
Roles & access	GET	`/projects/{project_id}/access-requests`	Pending and past requests	cap access.approve
Roles & access	POST	`/access-requests/{request_id}:decide`	Approve with roles or reject	cap access.approve (+ role.manage for non-observer roles)
Roles & access	GET	`/projects/{project_id}/gates`	Stage gate policies	cap arena.read
Roles & access	PUT	`/projects/{project_id}/gates/{stage}`	Update one gate policy	cap project.settings
Roles & access	GET	`/projects/{project_id}/agent-policy`	Agent autonomy policy (effective, after org/unit ceiling)	cap arena.read
Roles & access	PUT	`/projects/{project_id}/agent-policy`	Change agent autonomy	cap agent.configure
Arena	GET	`/projects/{project_id}/arena`	Arena and central-agent session status	cap arena.read
Arena	GET	`/projects/{project_id}/arena/messages`	Arena messages (single stream, threads via parent_id)	cap arena.read
Arena	POST	`/projects/{project_id}/arena/messages`	Post; @central starts a turn; @user/@role notify	cap arena.post (+agent.invoke for @central, +arena.tag_roles for @role)
Arena	PATCH	`/messages/{message_id}`	Edit own message	author
Arena	DELETE	`/messages/{message_id}`	Delete own message (soft)	author OR cap project.settings (moderation)
Arena	POST	`/projects/{project_id}/arena/turns/{turn_id}:cancel`	Cancel an agent turn	turn requester OR cap project.settings
Routines	GET	`/projects/{project_id}/routines`	Routines	cap arena.read
Routines	POST	`/projects/{project_id}/routines`	Create routine (min interval 1 hour)	cap routine.manage
Routines	PATCH	`/routines/{routine_id}`	Edit / pause routine	cap routine.manage
Routines	DELETE	`/routines/{routine_id}`	Delete routine	cap routine.manage
Routines	POST	`/routines/{routine_id}:run-now`	Run once now	cap routine.manage
Private	GET	`/private/sessions`	My private sessions	owner
Private	POST	`/private/sessions`	New private session (optionally bound to a project at creation)	authenticated (+cap arena.read on bind_project_id)
Private	GET	`/private/sessions/{session_id}`	Session	owner
Private	PATCH	`/private/sessions/{session_id}`	Rename	owner
Private	DELETE	`/private/sessions/{session_id}`	Delete session (and its runtime session)	owner
Private	GET	`/private/sessions/{session_id}/messages`	Messages	owner
Private	POST	`/private/sessions/{session_id}/messages`	Send to personal agent (starts a turn)	owner
Private	POST	`/private/sessions/{session_id}:bind`	Bind to exactly one project (write-once)	owner + cap arena.read on project
Private	POST	`/private/sessions/{session_id}/bridge/pull-context`	Bring a read-only snapshot of the bound project's context	owner + cap arena.read on bound project
Artifacts	GET	`/artifacts`	List artifacts (scope=private: mine; scope=project: needs arena.read)	owner / cap arena.read

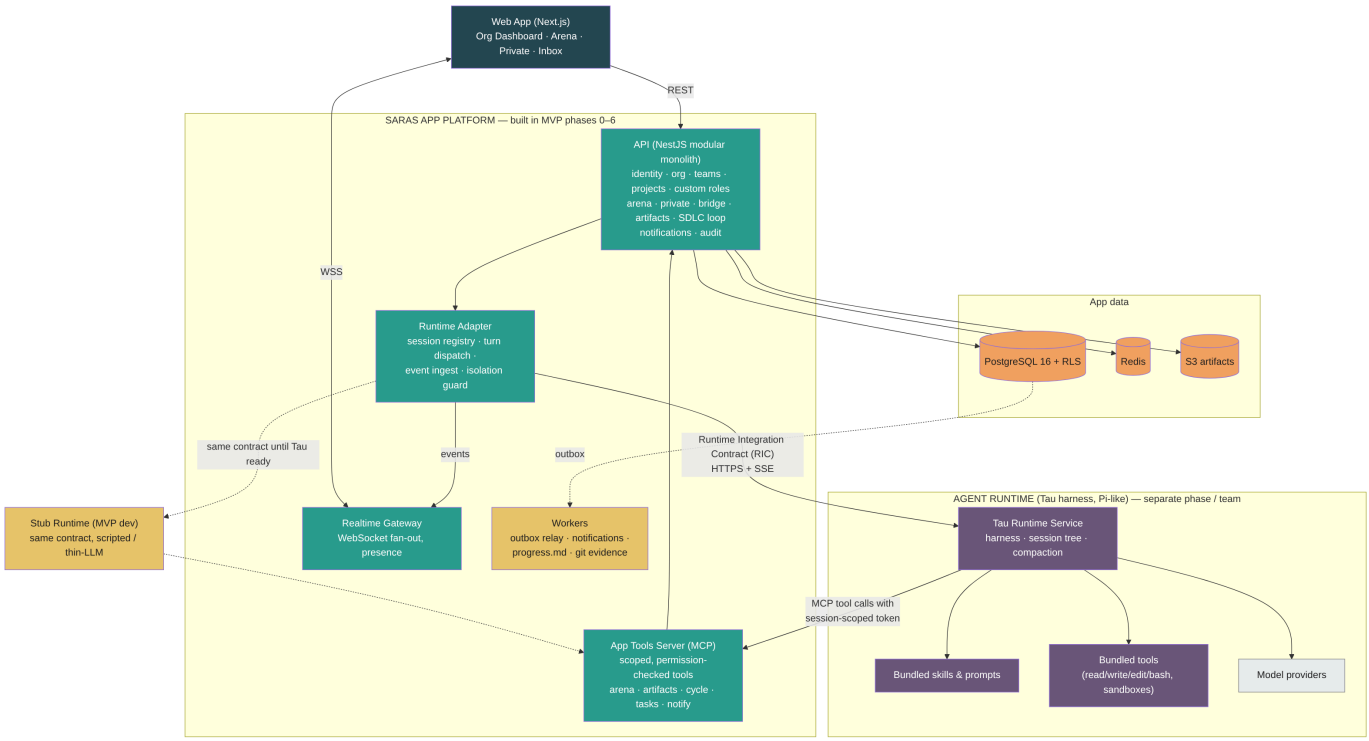
Tag	Method	Path	Summary	Required permission
Artifacts	POST	`/artifacts`	Create artifact (private, or project draft)	owner / cap artifact.edit
Artifacts	GET	`/artifacts/{artifact_id}`	Artifact metadata	owner / cap arena.read
Artifacts	GET	`/artifacts/{artifact_id}/versions`	Versions	owner / cap arena.read
Artifacts	POST	`/artifacts/{artifact_id}/versions`	New version	owner / cap artifact.edit
Artifacts	GET	`/artifacts/{artifact_id}/versions/{version_no}/content`	Raw content	owner / cap arena.read (export needs artifact.export)
Artifacts	GET	`/artifacts/{artifact_id}/push-preview`	Preview Push to Arena	owner + cap artifact.push on project
Artifacts	POST	`/artifacts/{artifact_id}/push`	Push a private artifact version to an Arena (binds the source session)	owner + cap artifact.push (+stage.request_gate if request_gate)
Artifacts	POST	`/files`	Pre-signed upload URL	authenticated (+cap on target scope)
Artifacts	POST	`/files/{file_id}:complete`	Mark upload complete (virus scan follows)	uploader
Tasks	GET	`/projects/{project_id}/tasks`	Tasks	cap arena.read
Tasks	POST	`/projects/{project_id}/tasks`	Create task	cap task.create (+task.assign if assignee set)
Tasks	GET	`/tasks/{task_id}`	Task	cap arena.read
Tasks	PATCH	`/tasks/{task_id}`	Update task (completion only via :verify)	assignee OR cap task.assign
Tasks	POST	`/tasks/{task_id}:delegate`	Delegate to a person or a role	cap task.delegate
Tasks	POST	`/tasks/{task_id}:assign-agent`	Assign to the AI teammate (needs a human reviewer)	cap agent.assign_task + agent policy assigned_work ≠ off
Tasks	POST	`/tasks/{task_id}:verify`	Verify acceptance criteria; close when all verified (human only)	cap task.verify (not the agent; not self if reviewer rule)
SDLC loop	GET	`/projects/{project_id}/cycles`	Cycles	cap arena.read
SDLC loop	POST	`/projects/{project_id}/cycles`	Open next cycle from MAINTAIN (one active cycle at a time)	cap cycle.open
SDLC loop	GET	`/cycles/{cycle_id}`	Cycle detail: gates, artifacts, pending approval, transitions	cap arena.read
SDLC loop	POST	`/cycles/{cycle_id}/gate-requests`	Request the current stage's gate	cap stage.request_gate
SDLC loop	POST	`/cycles/{cycle_id}:send-back`	Send back to an earlier stage (reason required)	cap stage.approve.
SDLC loop	GET	`/cycles/{cycle_id}/evidence`	Evidence (commits, PRs, CI, deployments, links)	cap arena.read
SDLC loop	POST	`/cycles/{cycle_id}/evidence`	Attach evidence link	cap artifact.edit
SDLC loop	POST	`/projects/{project_id}/repo:connect`	Link a GitHub repo (GitHub App installed by org admin)	cap project.settings
SDLC loop	GET	`/approvals`	Approvals I can vote on (mine=true)	authenticated
SDLC loop	GET	`/approvals/{approval_id}`	Approval	cap arena.read
SDLC loop	POST	`/approvals/{approval_id}/votes`	Vote (human only; SoD rules enforced)	cap stage.approve. (gate) / capability named in approval (agent action)
Messaging	GET	`/dms`	My DM threads	authenticated

Tag	Method	Path	Summary	Required permission
Messaging	POST	`/dms`	Open or reuse a DM thread (guests: only with co-members)	org member
Messaging	GET	`/dms/{thread_id}/messages`	DM messages	participant
Messaging	POST	`/dms/{thread_id}/messages`	Send DM	participant
Notifications	GET	`/notifications`	Notifications (filter by type / requires_action)	authenticated
Notifications	PATCH	`/notifications/{notification_id}`	Mark read / resolved	recipient
Notifications	POST	`/notifications:mark-all-read`	Mark all read	authenticated
Notifications	GET	`/notification-prefs`	Delivery preferences per type	authenticated
Notifications	PUT	`/notification-prefs`	Update preferences	authenticated
Notifications	POST	`/realtime/token`	5-minute WebSocket token	authenticated
Search	GET	`/search`	Permission-aware search (projects, messages, artifacts, tasks, people, my private sessions)	authenticated
Webhooks	POST	`/webhooks/workos`	WorkOS events (SCIM directory sync, connection changes); signature verified	public (signed)
Webhooks	POST	`/webhooks/github`	GitHub App events (push, PR, check runs) for evidence; HMAC verified	public (signed)

08 — Agent Runtime Integration (Tau)

> Build Pack v2.0 · Contracts: - `contracts/runtime-integration-contract_v0.2.yaml` (OpenAPI 3.1, Redocly-valid) - `contracts/app-tools_v0.2.mcp.json` (20 tools)
>
> Supersedes RIC v0.1.

8.1 Boundary



App and runtime boundary

Owned by the app	Owned by the runtime (Tau)
Who the agent is, which session it is in, which project it may touch	Harness, prompts, skills, model routing, compaction
Permissions, autonomy policy, approvals, audit	Bundled tools (read/write/edit/bash), sandboxes (post-MVP)
Persistence of messages, artifacts, tasks, turns, usage	Session state (tree), memory inside a session
Session keys and tool tokens	Honouring keys, tokens and isolation rules

Tau is the team's fork of the open-source **Pi** coding-agent harness. Pi provides interactive, print/JSON, RPC (JSONL over stdio) and SDK modes, tree-structured sessions with compaction, extensions and skills. The Tau team wraps the SDK in a small HTTP service that implements RIC. Saras' App Tools are loaded into Tau as one MCP extension.

Inside Tau, model routing is Tau's business. For example, Tau can use a cost-efficient primary model that escalates to a frontier model for planning and review. The app only sees `model_class` hints and usage.

8.2 RIC v0.2 at a glance

Operation	Endpoint	Notes
Health / capabilities	`GET /health`, `GET /capabilities`	Contract version, profiles (`central_arena`, `personal_private`), features (steering, follow-up queue, compaction, approvals, routines, assignments)
Create session	`POST /sessions`	Idempotent on `session_key`; `context_seed` = org brief, project brief, members with custom roles, gate summary; tool token
Bind private session	`POST /sessions/{id}:bind`	Write-once; new token scoped to the project

Operation	Endpoint	Notes
Rotate token	`PUT /sessions/{id}/token`	On membership change (immediately) and before expiry
Compact	`POST /sessions/{id}:compact`	Arena sessions are long-lived; Tau also compacts automatically
Turn	`POST /sessions/{id}/turns`	`turn_id` idempotent; `mode` = prompt / steer / follow_up / system / **routine** / **assignment**; `autonomy` (effective policy); `task_ref` / `routine_ref`; `app_state` (cycle, stage)
Events	`GET …/turns/{turn}/events` (SSE)	`turn.started`, `message.delta`, `message.completed`, `step`, `tool.called`, `tool.result`, `approval.requested`, `artifact.proposed`, `context.compacted`, `turn.completed` (usage), `turn.failed`, `turn.cancelled`
Cancel	`POST …/turns/{turn}:cancel`	—
Resolve approval	`POST /sessions/{id}/approvals/{aid}`	Human decision relayed by the app

Isolation rules (MUST, certified by the conformance kit): 1. No state, memory, cache or compaction shared between sessions. 2. A session never receives content for any project other than its scope or binding. 3. A token is used by one session only. 4. `turn_id` is idempotent. 5. No secrets or tokens appear in events.

8.3 Turn modes

Mode	Sent when	Runtime must
`prompt`	`@central`, Private message	Answer; may use tools within autonomy
`steer` / `follow_up`	User adds input while a turn runs	Inject now / queue next
`system`	Stage changed, member joined/left, routine config changed, ambient batch (`reply_expected=false`)	Update context; reply only if `reply_expected`
`routine`	Scheduler	Call `routine.context`, then post the result with `arena.post_message` (respect `post` autonomy)
`assignment`	Task assigned to the AI teammate	Work on `task_ref`. Produce drafts and comments, then `tasks.update_status(in_review)`. Never claim completion

8.4 App Tools v0.2 (20)

Tool	Autonomy class	Side effect	Notes
`arena.read_history`, `project.get_overview`, `members.list`, `cycle.get_state`, `artifacts.list`, `artifacts.read`, `tasks.list`, `knowledge.search`, `routine.context`	answer	read	Scoped to `token.project_id`
`arena.post_message`	post	write	Central agent only
`notify.tag`	notify	write	Tags members or roles
`artifacts.create_draft`	draft_artifacts	write	Drafts never auto-approve
`private.save_artifact`	—	write	Personal agent, Private only
`bridge.propose_push`	—	write-confirm	Creates a confirmation card; the user pushes
`tasks.create_draft`, `delegation.suggest`	tasks	write / confirm	A human accepts
`tasks.update_status`, `tasks.comment`	assigned_work	write	`update_status` only on the agent's own assignment; never `completed`
`stage.request_gate`	request_gate	write	Creates approval; humans vote
`approval.request`	(any)	write	For actions the policy marks `approve`

Never exposed to any agent: - vote on approvals; - verify or complete tasks; - manage members, roles or access requests; - change gate or agent policy; - manage routines; - push to an Arena without user confirmation; - archive projects; - send DMs.

8.5 Autonomy enforcement (two layers)

1. **Runtime layer:** the turn carries the effective `autonomy`. Tau should not attempt actions above it and should call `approval.request` for `approve`.
2. **App layer (authoritative):** App Tools re-check every call against the same policy and the on-behalf-of user's capabilities. Above-policy calls get `APPROVAL_REQUIRED` (with an approval created) or `FORBIDDEN`. A buggy or manipulated runtime therefore cannot exceed policy.

8.6 Stub runtime (app-team owned)

Mode	Use
`scripted`	Deterministic fixtures for CI and E2E: replies, tool calls, approvals, artifacts, failures
`thin-llm`	One model call per turn with App Tools wired in. Used for the YC demo, staging before Tau, and production fallback

Both implement RIC v0.2 completely. Switching to Tau is a config change (`agent_sessions.runtime_kind`, env `RUNTIME_URL`).

8.7 Conformance and isolation certification

Suite	Contents	Gate
RIC conformance	Lifecycle, idempotency, SSE resume, cancel, approval round-trip, bind write-once, token rotation, error codes, latency	≥95% before staging; 100% before GA
Canary isolation	Unique canary strings per project, private session and DM; adversarial prompts ("list other projects", "what did Alice write privately?"); output scan across every other session	100%, release-blocking
Autonomy	Each tool at each autonomy level, including `approve` round-trips	100%

8.8 Integration checkpoints with the Tau team

Checkpoint	Date	Criterion	If missed
C1 Contract frozen	29 Oct 2026	RIC v0.2 + App Tools v0.2 signed by both leads	App continues on the stub; changes go through an RFC
C2 Tau conformance vs stub App Tools	17 Dec 2026	≥95% conformance, 100% canary	P5 slips; the stub continues
C3 Tau alpha on staging	14 Jan 2027	Golden path on Tau	Beta launches on stub `thin-llm`; Tau follows in MVP+1
C4 Tau beta	5 Feb 2027	7 days on staging within SLOs	GA on whichever runtime is certified

09 — UX Specification & Design System

> Build Pack v2.0 · Visual source of truth: design canvas **"Saras App UI"** (14 artboards, light default, dark toggle per artboard). Built from the founders' Private-mode inspiration: pill mode switcher, soft violet accent, rounded composer, sidebar structure.

9.1 Information architecture

```
Global header:  [Private | Arena | Developer (soon)]   Search   Notifications   Profile
Private (global, also a dock on every screen)
  ├─ New session · My artifacts · My Arenas · Recent sessions (project tag when bound)
  └─ Session: conversation · bridge (bring context / push) · session context rail
Arena
  ├─ Sidebar: Inbox · New project · My Arenas · Direct messages
  ├─ Project: Arena (single stream) · Loop · Tasks · Artifacts · Routines · Settings (Members & roles, Gates, Agent,
Access requests)
  └─ Setup wizard (7 steps)
Organization (by role)
  ├─ Dashboard (org / unit scope)
  ├─ Structure (units, unit roles) · Members & invites · Teams
  ├─ Org knowledge · Policies · SSO & SCIM · Agent usage · Audit log
Inbox & DMs
Onboarding
```

9.2 Screen inventory (canvas artboards)

#	Artboard	Purpose	Key states to build
1	Private · Home	Empty state, composer, recent sessions	Empty / with history; model selector; "not bound" chip
2	Private · Session	Conversation, artifact card, bridge rail	Unbound / bound; agent streaming; failure; long history
3	Private · Push to Arena	Preview and confirm dialog	New vs new version; gate request; locked destination; error
4	Arena · Project space	Single stream, loop strip, approval card, right panel	Member / Observer (read-only banner) / agent degraded / empty Arena
5	Arena · Private dock	Private as slide-over with context picker	Picker; bind warning; existing bound session
6	Arena · SDLC loop	Cycles, stage cards, gate decision, history, progress.md	Pending / approved / sent back; SoD error; MAINTAIN + open next cycle
7	Setup · Custom roles	Templates, role × capability matrix, assignment	Template switch; blank; validation errors
8	Setup · Stage gates	Per-stage mode, approver roles, min approvals	Valid; GATE_UNSTAFFED; DISTINCT_ROLES_UNSATISFIABLE
9	Org admin · Dashboard	KPIs, project table, stage distribution, knowledge	Org scope / unit scope (Unit Lead); empty org
10	Notifications & DMs	Filters, action-needed, DM thread	Empty; many unread; DM with guest
11	Onboarding	Create organization (preset)	Startup / Enterprise preset; slug taken
12	**Org structure & roles** (v2)	Unit tree, unit roles, org roles, people	Add unit; assign Unit Lead; guest with expiry
13	**Project settings · Members & access** (v2)	Members with roles, Observer, access requests, agent autonomy	Approve request; last-owner guard; autonomy capped by org
14	**Routines & AI teammate** (v2)	Routine list with last run, task assigned to AI with reviewer	Routine failed; awaiting approval; task in review

9.3 Design tokens

Token	Light	Dark	Use
`--bg`	#F6F6F7	#0B0B0D	App ground

Token	Light	Dark	Use
`--side`	#EFEFF1	#111114	Sidebars
`--panel`	#FFFFFF	#151518	Cards, panels, composer
`--raise`	#F2F2F4	#1E1E22	Hover, chips, quiet fills
`--line` / `--line2`	#E3E3E7 / #D3D3D9	#26262B / #36363C	Borders
`--text`	#17171B	#F1F1F3	Body
`--muted`	#55555F	#A6A6B0	Secondary text
`--faint`	#6A6A74	#90909A	Meta text (≥4.5:1 on panel)
`--acc`	#6A55E3	#9B88FF	Primary actions, focus, agent identity
`--acc-ink`	#5541D3	#B6A8FF	Accent text
`--acc-soft` / `--acc-line`	#EFECFD / #D7D0FA	#221D3E / #3B3268	Agent message background, selected states
`--on-acc`	#FFFFFF	#100C22	Text on accent
`--ok` / `--ok-soft`	#1B7A46 / #E4F3EA	#4FC48C / #11271B	Approved, verified, online
`--warn` / `--warn-soft`	#94570A / #FBF0DC	#E5AB4E / #2A2011	Waiting on gate, away
`--bad` / `--bad-soft`	#B0341D / #FBE6E1	#F27D65 / #2D1611	Blocked, rejected

- **Typography:** Geist (UI), Geist Mono (keys, file names, ids).
- **Type scale:**

Use	Size
Display	56 px
H1	26 px
H2	16–19 px
Body	14 px
Meta	12 px

- **Radius:** 10 px (buttons), 14–16 px (cards), 18 px (composer), 999 px (pills, chips, avatars).
- **Spacing:** 4-px base; page gutters 24–40 px; 16 px minimum on mobile.
- **Elevation:** one soft shadow for composer, dialogs and popovers only.

9.4 Component inventory

Component	Variants	Notes
Mode switcher (pill)	Private / Arena / Developer (disabled “Soon”)	Matches inspiration
Button	primary, secondary, ghost, icon (44 px target)	Focus ring = `--acc` 2 px
Chip	neutral, accent, ok, warn, bad, mono key	Roles, statuses, project keys
Avatar	person, agent (sparkle on accent-soft), with presence dot	Agent never uses a human-looking avatar
Composer	Private (model selector, bind chip), Arena (tag menu, audience line), DM	No voice button in MVP
Message	human, agent (accent-soft bubble, “Agent” chip, cited sources, steps button), event row, routine post, approval card	Agent always labelled
Approval card	gate, agent action	Shows approver roles, progress (n of m), SoD hint
Artifact card	private, project; version chip; actions	Push to Arena on private cards
Loop strip / stage card	6 stages	Current stage in accent
Role chip and role × capability matrix	—	Owner locked to all
Notification row	action-needed, info	Deep link
Dock (Private slide-over)	—	460 px, focus-trapped
Wizard stepper	7 steps	Completed / current / upcoming
Data table (grid-based)	dashboard projects, members	Horizontal scroll on small screens

9.5 Interaction rules

1. **Always show where you are and who can see it:**
   - composer audience line (“everyone in IPP can see this” / “Only you”);
   - project binding chip in Private.
2. **Agent output is never mistaken for a human’s:** accent bubble, “Agent” chip, sparkle avatar.
3. **Approvals happen in place:** cards in the Arena and the Loop tab, plus inbox items. Rejecting or sending back requires a reason.
4. **Destructive or crossing actions are previewed:** push to Arena, decommission, remove member, binding a session.
5. **Read-only states are explicit:** the Observer banner replaces the composer and explains how to request a role.
6. **Errors say what to do next:** e.g. “DEPLOY needs an SRE approver, but no one has the SRE role. Add someone to SRE or change the gate.”

9.6 Copy guidelines

- Use product nouns consistently: **Arena**, **Private**, **Push to Arena**, **central agent** (UI: “Central agent”), **AI teammate**, **cycle**, **gate**, **routine**, **Observer**.
- Sentence case, short verbs (“Request gate”, “Assign to AI teammate”, “Bring context”).
- Never say “public” for Arena.
- Avoid jargon such as “RLS”, “capability” or “tenant” in the UI. Say “permission” and “what this role can do”.

9.7 Accessibility and responsive

- WCAG 2.1 AA: text contrast ≥ 4.5:1 (tokens validated); 44 px touch targets; visible focus; Radix primitives for dialog, menu and tabs; live region for streaming agent text; `aria-label` on icon buttons.
- **Breakpoints:**

Width	Layout
≥ 1280 px	Three panes
1024–1279 px	Right panel collapses to a drawer
768–1023 px	Sidebar collapses to icons
< 768 px	Single column with bottom navigation; reading, approving and messaging supported. Setup wizard and role matrix are desktop-first

10 — Security, Privacy & Compliance

> Build Pack v2.0 · Supersedes v1.0 doc 07 where they differ.

10.1 Security principles

1. **The backend and database enforce boundaries.** The UI and prompts never do.
2. **Private is private.** No role, admin or agent can read another person's Private space.
3. **Oversight is not access.** Dashboards show status, never content.
4. **AI is a least-privileged delegate.** Agent permission = human ∩ project autonomy ∩ org ceiling ∩ invariants.
5. **Every consequential action is attributable.** Actor, on-behalf-of, decision, reason. Append-only.

10.2 Threat model (top risks)

#	Threat	Mitigations	Residual
T1	Cross-tenant data leak	FORCE RLS on 41 tables; org-prefixed cache keys; tenant suite in CI	Low
T2	Private content exposed (to team, admin or agent)	RESTRICTIVE RLS; bridge is the only path; push copies the artifact only; canary suite	Low
T3	Agent reads or acts on the wrong project	Session keys, write-once bind, scoped tool tokens, App Tools scope check, canary suite	Low
T4	Prompt injection makes the agent act beyond intent	Autonomy policy enforced in App Tools (not in the prompt); human-only invariants; approvals for risky classes; org ceiling	Medium → Low
T5	Privilege escalation via custom roles	Only `role.manage` holders edit roles; ≥1 owner; capability catalogue fixed in code; audit of every role change	Low
T6	Self-approval / collusion on releases	`forbid_self_approval`, `min_approvals`, `min_distinct_roles`; votes audited	Low
T7	Stale access after people leave or move	SCIM deprovisioning (sessions revoked < 60 s); guest expiry; mover notices to team owners	Low
T8	Credential / token theft	WorkOS sessions; HttpOnly cookies; tool tokens ≤60 min bound to one session; secrets in AWS Secrets Manager; log redaction	Low
T9	Denial of wallet (runaway agent use)	Per-org budgets, per-user turn rate limits, routine min interval 1 h, ambient batching	Low
T10	Malicious file upload	Pre-signed uploads, type and size checks, virus scan, no inline HTML rendering	Low
T11	Webhook spoofing (WorkOS / GitHub)	Signature verification, delivery-id dedupe	Low

10.3 Identity and access

- **Login and sessions:**
  - WorkOS AuthKit (Google, Microsoft, magic link) and enterprise SSO (SAML / OIDC).
  - `require_sso` policy per org.
  - Sessions: 12 h absolute / 2 h idle. Admin screens require re-authentication after 30 min.
- **SCIM via WorkOS Directory Sync:** users, deprovisioning, groups → units (P1 within the MVP window).
- **RBAC:** org roles, unit roles, team roles and project roles with 27 capabilities, as defined in doc 03.

- **Break-glass:** there is no admin path to Private content in the MVP. Legal-hold export requires two org owners and is post-MVP.

10.4 Data protection

Data	At rest	In transit	Notes
PostgreSQL	RDS encryption (KMS)	TLS 1.2+	PITR 14 days
S3 artifacts / files	SSE-KMS, versioning	TLS	Audit archive in Object Lock
Redis	Encryption at rest + in transit	TLS	No content, ids only
Secrets	AWS Secrets Manager	—	Rotated quarterly
Model traffic	—	TLS	Runtime uses providers with zero-retention terms; content never used for training

Data region: ap-south-1 (Mumbai) for the MVP. Others on demand.

10.5 Application security controls

Control	Implementation
Input validation	Zod on every boundary; size limits (message 20 KB, artifact 2 MB, file 100 MB)
Output encoding	Markdown sanitised (`rehype-sanitize`); no raw HTML from users or agents
CSRF	SameSite cookies + double-submit token
CSP	Strict CSP with nonces; `frame-ancestors 'none'`
SSRF	No server-side URL fetch in the MVP app; runtime egress controlled in Tau
Dependencies	Renovate, `npm audit`, SBOM per release, signed images
Static analysis	Semgrep + CodeQL on PRs; gitleaks
Testing	RLS / tenant suite, canary isolation suite, authz matrix tests (release-blocking)
Pen test	External test before GA (focus: tenancy, Private, RBAC, App Tools)

10.6 Privacy

- **Data minimisation:** analytics events carry ids, never content.
- **Personal data:** names, emails, titles, levels, reporting lines (directory only).
- **Data-subject requests:** export of a user's Private content and profile (P1), and deletion via offboarding plus retention.
- **India DPDP Act and GDPR readiness:** DPA template, sub-processor list (AWS, WorkOS, model providers via Tau, PostHog, Sentry, GitHub), privacy notice. All before beta.

10.7 Compliance roadmap

Milestone	When
Security baseline + policies (access, change, incident, vendor)	Before beta (Feb 2027)
SOC 2 Type I (automation via Vanta / Drata)	GA + 3 months
SOC 2 Type II	GA + 9–12 months
ISO 27001	When enterprise pipeline requires

Customer benefit: the Saras audit trail (stage artifacts → gate votes → evidence) supports customers' own change-management controls.

10.8 Security checklist per feature (Definition of Done)

- ☐ Which plane(s) authorise it? Capability added to `x-required-permission`?
- ☐ RLS covers new tables (tenant + visibility)?
- ☐ Agents: is the action human-only? If not, which autonomy class?
- ☐ Can it leak Private or another project's content (UI, notifications, search, previews)?
- ☐ Audited if risk ≥ medium?
- ☐ Rate-limited / budgeted if it triggers model calls?

11 — Engineering Handbook

> Build Pack v2.0 · How we build Saras day to day: setup, standards, Definition of Done, testing, CI/CD, environments, observability and runbooks.

11.1 Day-1 setup

```
git clone git@github.com:<org>/saras.git && cd saras
corepack enable && pnpm install
cp .env.example .env.local            # WorkOS test keys, local S3 (MinIO), stub runtime URL
docker compose up -d                  # postgres16+pgvector, redis, minio, mailpit, stub-runtime
pnpm db:migrate && pnpm db:seed       # schema_v2 + demo org "Adva AI Lab" (startup + enterprise presets)
pnpm dev                              # web :3000, api :4000, realtime :4100, worker, app-tools :4200
```

Seed data: - 2 orgs: - “Adva AI Lab” (startup); - “Acme Global” (enterprise, 3 units). - 4 teams. - 5 projects across all templates. - 12 users, covering every org role, a Unit Lead, a guest and an observer.

Log in via the WorkOS test identity provider.

11.2 Coding standards

Topic	Rule
Language	TypeScript strict everywhere; no `any` without a comment
Contracts first	Change `packages/contracts` (Zod) → regenerate OpenAPI → review → implement
Module boundaries	`eslint-plugin-boundaries`: apps import packages, modules don’t import each other’s internals
Domain logic	Pure functions in `packages/domain` (state machines) and `packages/authz`; no DB inside
Database	Kysely only; every query runs inside `withTenant(tx, ctx)`; no raw string SQL concatenation
Errors	Throw typed `DomainError(code)` → mapped to problem+json; never leak stack traces
Logging	Structured (pino) with `request_id`, `org_id`, `user_id`, `turn_id`; **never** log message/artifact content, prompts or tokens
Frontend	Server state in TanStack Query; UI state in Zustand; components from `packages/ui`; no inline colors (tokens only)
Accessibility	Every interactive element is a real button, link or input; axe check in Playwright
Feature flags	PostHog flags for every user-visible new capability; flags removed within 2 sprints after GA

11.3 Git and review

- **Trunk-based:** short-lived branches `feat/F09-arena-mentions`. PRs under 400 lines where possible.
- **Conventional commits.** Squash merge.
- **One approval required.** Two for `packages/authz`, `packages/db/migrations`, `apps/app-tools`, and RLS policies.
- **PR template asks:**
  - feature ID and backlog story;
  - contract change?;
  - migration?;
  - security checklist (doc 10 §10.8);
  - screenshots for UI.

11.4 Definition of Done (every story)

- [ ] Acceptance criteria from the backlog met and demoed on staging
- [ ] API matches `openapi_v2.yaml` (contract test green); `x-required-permission` correct
- [ ] Authz tested (allowed and denied paths, including agent principal where relevant)

- [ ] RLS covers new tables; tenant and isolation suites green
- [ ] Audit events for medium/high-risk actions; notifications wired
- [ ] UI states: loading, empty, error, read-only, dark theme
- [ ] Analytics event (ids only) where listed in PRD §2.6
- [ ] Docs updated (this Build Pack) when behaviour differs

11.5 Testing strategy

Layer	Tooling	Must cover
Unit	Vitest	`packages/authz` (≥95% branches), `packages/domain` state machines, cron validation, progress formula
Integration	Vitest + Testcontainers (PG16, Redis, MinIO)	Use-cases with real DB + RLS; outbox → consumers; setup:complete atomicity
Contract	Schemathesis against staging; oasdiff on PRs	No breaking change without a version bump
Runtime	RIC conformance kit vs stub (every PR) and vs Tau (checkpoints)	Lifecycle, idempotency, SSE resume, approvals, bind
Security	Tenant suite (every table with foreign org id), Private suite, canary isolation, authz matrix	Release-blocking
E2E	Playwright	Golden paths: onboarding → setup (custom roles) → Arena → Private push → gate → AI teammate → routine → dashboard
Load	k6	500 concurrent Arena users, 50 msg/s, 100 concurrent turns (stub with latency)

11.6 CI/CD

```
PR:     lint · typecheck · unit · integration (Testcontainers) · contract (oasdiff) · migrations on fresh DB
        · RLS/tenant + authz suites · RIC conformance vs stub · Semgrep/CodeQL/gitleaks · build images
main:   deploy staging (ECS blue/green) → Playwright golden paths → k6 smoke
release tag vX.Y.Z: manual approval (we dogfood our own DEPLOY gate) → prod blue/green → automatic rollback on error-rate
alarm
```

11.7 Environments and configuration

Variable	Purpose
`DATABASE_URL`, `REDIS_URL`, `S3_BUCKET_*`	Data stores
`WORKOS_API_KEY`, `WORKOS_CLIENT_ID`, `WORKOS_WEBHOOK_SECRET`	Auth / SSO / SCIM
`RUNTIME_URL`, `RUNTIME_KIND` (stub | tau), `RUNTIME_JWT_KEY`	Agent runtime
`APP_TOOLS_URL`, `TOOL_TOKEN_KEY`	MCP server and token signing
`GITHUB_APP_ID`, `GITHUB_PRIVATE_KEY`, `GITHUB_WEBHOOK_SECRET`	Evidence
`SES_*`, `POSTHOG_KEY`, `SENTRY_DSN`, `OTEL_EXPORTER_OTLP_ENDPOINT`	Email, analytics, errors, tracing

11.8 Observability and SLOs

SLI	SLO	Alert
API availability	99.9% / 30 d	Burn rate 14× / 1 h, 6× / 6 h
API p95 latency	< 250 ms reads	> 400 ms for 10 min
Arena fan-out p95	< 300 ms	> 1 s for 5 min
Agent first event p95	< 3 s	> 6 s for 10 min
Turn success rate	> 97%	< 93% for 15 min
Outbox lag	< 5 s	> 30 s
Routine on-time rate	> 99% within 2 min of schedule	< 95% / day
Isolation alarms	0	Any `SCOPE_MISMATCH` from adapter guard → page

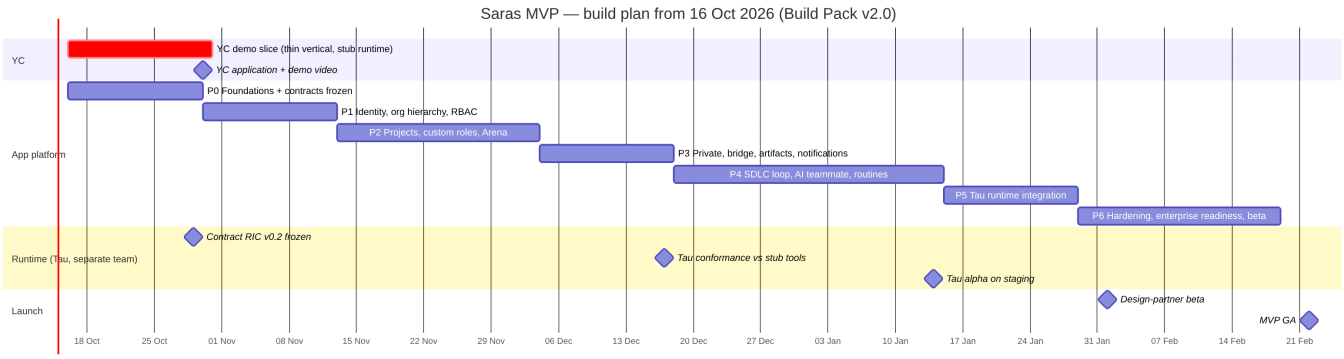
11.9 Runbooks (summary)

Situation	First steps
Runtime down	Confirm via `/health`. The app auto-degrades. Post a status banner. Routines fall back to templates. Switch `RUNTIME_KIND=stub` (thin-llm) if the outage is > 30 min
Isolation alarm	Freeze the affected sessions (`agent_sessions.status=failed`). Rotate tool tokens. Collect `app_tool_calls` and audit. Incident SEV1
WorkOS outage	Existing sessions continue. New logins fail. Status banner
GitHub outage	Evidence jobs retry for 24 h. Gates are unaffected
Bad deploy	ECS rollback to the previous task set. Migrations are expand-only, so rollback is safe
Budget exhausted	Org notified at 80/100%. A Billing admin raises the budget. Turns return `BUDGET_EXCEEDED` until then

12 — Delivery Plan, YC Demo Slice & Backlog

> Build Pack v2.0 · Start: **Friday 16 Oct 2026** · MVP GA target: **Monday 22 Feb 2027** Detailed stories: `Saras_MVP_Backlog_v2.xlsx` (77 stories · 365 points). Capacity sheet: 6 engineers × 8 pts × 9 sprints = 432 pts → 84% planned utilisation.

12.1 Timeline



Roadmap

Phase	Dates	Outcome	Exit criteria (demo on staging)
YC demo slice	16 – 30 Oct	Thin end-to-end slice for the application video	Login → org → project with custom roles → Arena chat with `@central` (stub thin-LLM) → Private draft → Push to Arena → gate approve. Recorded 3-min video (doc 01 §1.9)
P0 Foundations	16 – 29 Oct	Monorepo, CI, Terraform, schema_v2, RLS harness, request pipeline, outbox, design system, stub runtime	Fresh `terraform apply` + `pnpm dev`; tenant suite green; RIC v0.2 frozen (C1)
P1 Identity, hierarchy, RBAC	30 Oct – 12 Nov	WorkOS auth/SSO, orgs & presets, policies, units & unit roles, members/invites/guests/offboarding, teams, authz library, audit	MNC demo: enterprise org with 3 units, Unit Lead, guest with expiry; authz matrix tests ≥95% branches
P2 Projects, roles, Arena	13 Nov – 3 Dec	Setup wizard (templates, custom roles, gates, agent), members & roles, observers, Arena realtime, `@central` on stub, App Tools, canary suite v1	Create "Enterprise product squad" project with custom role; observer read-only; canary 100%
P3 Private, bridge, artifacts, notifications	4 – 17 Dec	Private sessions, bind / pull / push, artifacts & versions, delegation, notification centre, DMs	Bind write-once demo; push with gate request; 10 notification types
P4 Loop, AI teammate, routines	18 Dec – 14 Jan	Cycles, gates with SoD, evidence, progress.md, AI-teammate tasks, routines, access requests, dashboards	Full cycle with 2-approver DEPLOY, no self-approval; AI task → review → verify; daily digest posts; Unit Lead dashboard
P5 Tau integration	15 – 28 Jan	Swap stub → Tau, ambient batching, step drawer, usage & budgets	Conformance ≥95%, canary 100%, p95 first event < 3 s (C3 by 14 Jan)
P6 Hardening & beta	29 Jan – 18 Feb	Load, pen test, search, SCIM, observability, email digest, audit UI; design-partner beta from 1 Feb	Launch checklist (§12.6) green
GA	22 Feb 2027	—	—

The holiday period (late December) is inside P4, so P4 runs 4 weeks.

12.2 YC demo slice — day by day (16–30 Oct)

Day	Date	Backend	Frontend	Platform
1	Fri 16	Repo, NestJS skeleton, schema_v2 migration, seed	Next.js shell, tokens, mode switcher	AWS account, Terraform base, CI

Day	Date	Backend	Frontend	Platform
2–3	Mon 19 – Tue 20	WorkOS login; orgs; teams (minimal)	Login, onboarding (org step)	Staging ECS + RDS
4–5	Wed 21 – Thu 22	Projects + setup:complete (templates, roles, gates; no units)	Setup wizard (roles + gates screens)	Stub runtime thin-LLM deployed
6–7	Fri 23 – Mon 26	Arena messages, realtime, `@central` turns via adapter	Arena screen (stream, approval card)	Seed demo org/data
8–9	Tue 27 – Wed 28	Private sessions, push to Arena, gate request + vote	Private home/session, push dialog	Monitoring basics
10	Thu 29	Bug bash; demo script dry-run	Polish	Freeze staging
11	Fri 30	**Record demo video**; submit YC application	—	—

Slice rules: - Code goes to `main` behind feature flags. - Shortcuts are listed in `SLICE_DEBT.md` and paid back in P1–P3 (e.g. no units, simplified authz checks enforced by RLS only, single org preset).

12.3 Team and ownership

Role	People	Owns
Tech lead / backend	1	Contracts, authz, loop, adapter, code review of security-critical paths
Backend	2	Org / units / teams / members; Arena / realtime / DMs; Private / bridge / artifacts / tasks / routines
Frontend	2	Design system, Private, Arena, setup wizard, loop, dashboards, inbox, settings
Platform / QA	1	Infra, CI/CD, observability, canary and tenant suites, load tests
Product / design (founder)	1	Scope, backlog, design canvas, design partners, YC application
Tau runtime team	Separate	Tau harness, RIC wrapper, conformance (doc 08 §8.8)

12.4 Ceremonies

- 2-week sprints starting Monday 19 Oct (the YC slice runs inside sprint 1).
- Daily 15-minute standup. **We dogfood:** run Saras itself in a Saras Arena as soon as P2 lands, with a daily digest routine.
- Sprint review = staging demo against the phase exit criteria.
- Weekly 30-minute contract sync with the Tau team.
- Architecture decisions recorded in doc 13 (append-only).

12.5 Risks

Risk	L	I	Mitigation	Owner
YC slice too ambitious for 2 weeks	M	H	Thin scope, stub runtime, RLS-first authz, explicit slice debt	Tech lead
RBAC complexity slows P1–P2	M	H	Authz as pure library with matrix tests; templates reduce UI surface	Tech lead
Tau slips	M	M	Stub thin-LLM is production-capable; checkpoints with fallbacks	Founder + Tau lead
Model cost	M	M	Budgets, batching, routine minimum interval, model classes	Backend
Enterprise asks (SAML/SCIM) early	M	M	WorkOS covers them; SCIM in P6	Backend
Scope creep (voice, Developer mode, channels)	H	M	Non-goals list (doc 02 §2.2); decision log	Founder

12.6 Launch checklist (GA)

- ☐ Golden-path E2E green for 7 consecutive days on staging
- ☐ Tenant, Private, canary isolation and authz suites 100%
- ☐ Unit Lead and Org Admin cannot read Arena content of non-member projects (verified in a prod-like environment)
- ☐ External pen test: no open High / Critical
- ☐ SLO dashboards and alerts live; runtime-down and isolation runbooks rehearsed
- ☐ Backup restore drill done (RPO ≤ 5 min, RTO ≤ 1 h)
- ☐ Privacy notice, DPA, sub-processor list published
- ☐ Budgets on for every org; billing plan configured
- ☐ Design partners' feedback triaged; P0 issues closed

12.7 Decisions to confirm with founders (defaults used in this pack)

#	Question	Default
D1	Can org admins read Arena content of projects they're not in?	No; they request access
D2	Team ↔ project	A team owns many projects; members can come from other teams/units if policy allows
D3	Parallel cycles	No; one active plus operating cycles
D4	Agent replies on someone's behalf	Not in MVP
D5	Rebinding a private session	Never; start a new session
D6	Hosting	AWS ECS Fargate, ap-south-1
D7	Auth vendor	WorkOS
D8	Plans and prices	After design-partner interviews

13 — Decision Log (ADRs, consolidated)

> Build Pack v2.0 · Append-only. A changed decision gets a new entry that supersedes the old one. Full option analyses for v1 ADRs remain in the v1.0 playbook doc 09.

ID	Decision	Status	Date	Alternatives considered	Revisit when
ADR-001	TypeScript modular monolith (NestJS) + separate realtime / worker / app-tools services	Accepted	12 Oct	Microservices; Python backend	Team > 15 or independent scaling needs
ADR-002	PostgreSQL 16 as system of record (+ pgvector, FTS)	Accepted	12 Oct	Postgres + OpenSearch + vector DB; MongoDB	> 50M vectors / DB CPU > 60%
ADR-003	Temporal for workflows	**Superseded by ADR-016**	12 Oct	—	—
ADR-004	Transactional outbox → Redis Streams	**Superseded by ADR-017**	12 Oct	—	—
ADR-005	Artifacts in S3 (content-addressed); Git as evidence mirror	Accepted	12 Oct	Git as primary store	—
ADR-006	One personal agent per user, one central agent per org; roles are modes, not identities	Accepted	12 Oct	Agent per role	Demand for “team agents”
ADR-007	Authz in-house (pure library + Postgres facts + RLS)	Accepted, extended by ADR-020	12 Oct	OpenFGA / SpiceDB; OPA / Cedar	Per-resource sharing, cross-org guests
ADR-008	Sandboxes in the app	**Superseded by ADR-014** (runtime owns sandboxes)	12 Oct	—	—
ADR-009	Own WebSocket gateway; REST is the only write path	Accepted	12 Oct	Ably / Pusher; SSE-only	Multi-region presence
ADR-010	Hybrid search in Postgres, ACL in SQL	Accepted	12 Oct	Dedicated search engine	With ADR-002 trigger
ADR-011	Multi-tenancy: shared DB + `org_id` + FORCE RLS	Accepted	12 Oct	Schema- or DB-per-tenant	Regulated single-tenant deals
ADR-012	Model gateway in the app	**Superseded by ADR-014**	12 Oct	—	—
ADR-013	Agent runtime is a separate product (Tau) integrated via a contract (RIC) + App Tools MCP	Accepted	12 Oct	Embed runtime in app	—
ADR-014	Tau owns harness, skills, tools, model routing and sandboxes; app owns identity, permissions, approvals, persistence	Accepted	12 Oct	Shared ownership	—
ADR-015	One Arena per project = one shared agent session; no channels; private sessions bind to ≤1 project, write-once	Accepted	12 Oct	Channels; per-user sessions in Arena	Users need sub-spaces (post-MVP threads first)
ADR-016	Long waits (gates, approvals) as DB state machines; jobs and schedules on **pg-boss**	Accepted	12 Oct	Temporal; Step Functions; Inngest	Complex multi-step orchestration in the app
ADR-017	Outbox → **pg-boss** consumers; Redis pub/sub only for realtime fan-out	Accepted	15 Oct	Redis Streams; Kafka	> 20 consumers or replay needs

ID	Decision	Status	Date	Alternatives considered	Revisit when
ADR-018	SDLC as a loop of cycles; one active development cycle per project	Accepted	12 Oct	Parallel intents	Teams need parallel streams
ADR-019	**Four permission planes** (org, unit, team, project) + agent plane; permissions follow responsibility, not seniority; oversight ≠ access	Accepted	15 Oct	Hierarchy-inherited access; fixed project roles	Enterprise feedback
ADR-020	Project roles = **team-defined bags of a fixed 27-capability catalogue** + system Owner/Observer roles; gate approvers point to roles	Accepted	15 Oct	Fixed PM/Eng/QA roles; free-form permissions	New capability needs (ship with migration, default off)
ADR-021	Agent **autonomy policy** per project per capability class (off / suggest / approve / act), capped by org and unit ceilings, enforced in App Tools	Accepted	15 Oct	Prompt-only rules; global switch	Usage data from beta
ADR-022	**Separation of duties** on gates: `min_approvals`, `min_distinct_roles`, `forbid_self_approval`	Accepted	15 Oct	Single approver	—
ADR-023	**WorkOS** for login, SSO (SAML/OIDC) and SCIM	Accepted	15 Oct	Auth.js + custom SAML; Auth0; Clerk	Pricing at scale
ADR-024	Hosting on **AWS ECS Fargate** (ap-south-1), Terraform	Accepted	15 Oct	EKS; Render / Fly	> 30 services or need for k8s-native tooling
ADR-025	**PostHog** for product analytics and feature flags; Sentry for errors; OTel → Grafana Cloud	Accepted	15 Oct	Unleash + Mixpanel	—
ADR-026	**Routines** run as `mode=routine` turns on the central agent's Arena session, with deterministic fallback	Accepted	15 Oct	Separate scheduler agent	—
ADR-027	**AI teammate** tasks: agent can reach `in_review` only; human reviewer required; DB check on `verified_by`	Accepted	15 Oct	Agent self-completion with audit	—
ADR-028	Org **presets** (Startup / Enterprise) set policy defaults; everything stays editable	Accepted	15 Oct	Single default	—

14 — Glossary

Term	Meaning
Organization (org)	The tenant: one company or group. Holds identity, policies, billing, audit, org knowledge
Org unit	Optional node in the org hierarchy (business unit, department, group), up to 5 levels
Unit Lead / Unit Admin	Unit-scoped roles: status dashboards for the subtree; team administration (Admin)
Org roles	Owner, Admin, Security & Compliance Admin, Billing Admin, Member, Guest
Guest	External person with expiring access to specific projects only
Team	Self-governed group of people that owns projects. Roles: Owner, Admin, Member
Project	A product, initiative or program. Has exactly one Arena, its own roles, gates, agent policy and routines
Arena	The project's single shared space: one message stream plus one central-agent session
Private space	A person's own AI workspace, available on every screen; never visible to others
Bridge	The only path between Private and an Arena: bind, pull context, push artifact
Bind	Tying a private session to exactly one project, permanently
Push to Arena	Copying one artifact version from Private into a project, with preview
Custom role	A team-named role made of capabilities (e.g. "Backend dev", "Security champion")
Capability	One of 27 fixed permissions (e.g. `arena.post`, `stage.approve.DEPLOY`)
Project Owner / Observer	System roles: Owner has every capability; Observer is read-only
Role template	Starting set of roles, gates and agent policy (7 templates)
Central agent	The project's AI participant in the Arena (one session per Arena)
Personal agent	A person's AI in Private, acting only on their behalf
AI teammate	The central agent working on an assigned task; output goes to human review
Autonomy policy	Per project, per capability class: off / suggest / approve (needs human approval) / act
Autonomy ceiling	Org (or unit) maximum autonomy that every project inherits
Routine	Scheduled AI job (daily digest, standup, gate nudge, weekly report, custom)
SDLC loop	PLAN → DESIGN → BUILD → TEST → DEPLOY → MAINTAIN → next cycle
Cycle	One pass through the loop; one active at a time, earlier ones keep "operating"
Gate	Approval needed to leave a stage; approvers are roles
Separation of duties (SoD)	Gate rules: minimum approvals, distinct roles, no self-approval
Stage artifact	goal.md, spec.md, plan.md, test report, release notes
Evidence	Proof attached to a stage: git commit, PR, CI run, deployment, link
progress.md	Summary generated from project records; never the source of truth
Tau	Saras' agent runtime: a fork of the Pi coding-agent harness
RIC	Runtime Integration Contract between the app and Tau (v0.2)
App Tools	MCP tools the app exposes to the runtime, scoped to one session's project
Stub runtime	App-team implementation of RIC (scripted / thin-LLM)
Canary suite	Tests that plant unique strings per project / session and prove they never leak
Turn	One agent run in a session (prompt, system, routine, assignment, ...)